DataOrbit AI — Product Blueprint

The Definitive Reference for What We're Building, Why, and How

Version: 1.0 **Date:** February 17, 2026 **Status:** Foundation Document — Living Reference

---

How to Use This Document

This document is the single source of truth for DataOrbit AI's product strategy, architecture, and build plan. It is designed for three audiences: founders making daily product decisions, engineers building the platform, and future investors or partners who need to understand the full picture.

It is structured in six parts:

1. **The Mission** — Why this company exists and what problem it solves
2. **The Market** — Who we serve, what they currently do, and why existing solutions fail them
3. **The Product** — What DataOrbit AI is, how it works, and what the customer experiences
4. **The Architecture** — How it's built, what technologies we use, and where the engineering risks are
5. **The Build Plan** — What we build first, second, and third — with honest scope and timelines
6. **The Business** — How we price, position, and grow

Each section is written to be self-contained but cross-referenced. Read it front to back for the full picture, or jump to a section for specific context.

Table of Contents

Part 1: The Mission

- 1.1 The Problem We Exist to Solve
  - The Reality: Most SMBs Run on Siloed SaaS Tools
  - The Problem: Cross-System Blindness
  - The Three Scenarios: It's Not Just "No Reporting"
  - The Common Thread: What Every Scenario Needs
  - What Needs to Exist
- 1.2 Why Now
- 1.3 The Company
- 1.4 The Positioning Statement

Part 2: The Market

- 2.1 Target Customer Profile
  - Primary ICP: The "Systemized but Siloed" SMB
  - Secondary ICP: The "Outgrowing Spreadsheets" Scale-Up
- 2.2 Target Verticals
- 2.3 The Competitive Landscape
  - Tier 1: Enterprise Cloud Data Warehouses
  - Tier 2: Managed SMB Warehouses
  - Tier 3: Enterprise Agentic Analytics
  - Tier 4: SMB Dashboard & KPI Tools
  - Tier 5: AI Chatbots
  - The True Competitor: Spreadsheets
- 2.4 Three Defensible Differentiators

Part 3: The Product

- 3.1 The Customer Experience
  - What the Customer Sees
  - What the Customer Does Not See
- 3.2 The Six-Layer Architecture
  - Layer 1: Managed Warehouse + Data Connectors
  - Layer 2: Data Quality Engine
  - Layer 3: Entity Matching Engine
  - Layer 4: Unified Entity Graph
  - Layer 5: Semantic Layer (Cube Core)
  - Layer 6: AI Agent — DataOrbit Apollo
- 3.3 Design Rationale: Why Each Layer Is Designed This Way
  - The Data Quality Engine: Why a Dedicated Layer, Not a Feature
  - The Unified Entity Graph: Why "Graph," Not "State Machine"
  - The Semantic Layer: Why Dramatically Simpler Than Enterprise Approaches
  - Influence Maps: Why Not "Causal Graphs"
  - The Pattern Intelligence Flywheel: Near-Term Feature, Long-Term Moat
  - The AI Agent: Why Synthesis Over Autonomy
  - The BI Integration: Why One Source of Truth Matters
- 3.4 BI Integration Layer

- 3.5 AI Capabilities Across Every Layer
    - Layer 1: Managed Warehouse + Data Connectors
    - Layer 2: Data Quality Engine
    - Layer 3: Entity Matching Engine
    - Layer 4: Unified Entity Graph
    - Layer 5: Semantic Layer (Cube Core)
    - Layer 6: AI Agent (DataOrbit Apollo)
    - Cross-Cutting AI Capabilities
    - AI Capability Roadmap by Phase

Part 4: The Architecture

- 4.1 Technology Stack
- 4.2 Data Flow Architecture
- 4.3 Data Modeling Philosophy: Traditional Approaches vs. DataOrbit
    - What Is the Same: Shared Foundations
    - What Is Different: Genuine Architectural Differences
    - What Is Enhanced: Improvements Over Traditional Approaches
    - What Has Been Productized: Consulting vs. Product
    - How to Talk About This
- 4.4 Template Architecture: How Pre-Built Intelligence Actually Works
    - The Problem With Monolithic Templates
    - The Composable Template Architecture (Levels 1-5)
    - Concrete Example: Two Companies, Different Systems, Same Outcome
    - What Breaks Down: Honest Assessment of the 20% Gap
    - How AI Closes the 20% Gap
    - The Revised Template Claim
    - What This Means for Engineering Priorities
- 4.5 Key Technical Risks and Mitigations
- 4.6 AI Implementation Architecture
    - The Four-Component Pattern
    - How Each Component Works (Intent Router, Context Fetcher, Action Executor, Response Writer)
    - Conversation State Management
    - Cost Model
    - Concrete Example: End-to-End Q&A Flow
    - The Query Pipeline: How Natural Language Becomes a Database Answer

- What Does NOT Need an LLM

Part 5: The Build Plan

- 5.1 Philosophy: Incremental, Customer-Driven
- 5.2 Phase 1: The Week One Briefing (Weeks 1-6)
- 5.3 Phase 2: Intelligence That Comes to You (Weeks 7-14)
- 5.4 Phase 3: The Full Platform (Weeks 15-26)
- 5.5 Phase 4: Scale and Expand (Months 7-12)

Part 6: The Business

- 6.1 Revenue Model
- 6.2 Go-to-Market Strategy
  - Phase 1: Founder-Led Sales (Months 1-6)
  - Phase 2: Content-Led Growth (Months 4-9)
  - Phase 3: Product-Led Growth (Months 7-12)
- 6.3 Key Metrics We Track
- 6.4 The Moat

Appendices

- Appendix A: Honest Limitations
- Appendix B: Competitive Quick-Reference
- Appendix C: Documents Referenced
- Appendix D: Future Work — Sections to Be Documented

PART 1: THE MISSION

1.1 The Problem We Exist to Solve

> Most SMB companies running 3-7 SaaS tools have no reliable and unified view of their customers, revenue, employees, or operations across systems. SMBs get enterprise-grade data intelligence that was previously only accessible to companies with dedicated data engineering teams.
> DataOrbit's Agentic Data Warehouse solves this — it builds a unified view across every connected system, and delivers a context-aware AI agent that provides real-time, trustworthy intelligence on demand. A fully managed, end-to-end solution — from data ingestion to agentic analytics — without needing a data team, a heavy technology footprint, or a big budget.

The Reality: Most SMBs Run on Siloed SaaS Tools

Most companies with 50-500 employees run on 3-7 SaaS tools. Each tool serves a department well on its own. Each generates data, reports, and dashboards within its own domain:

- **CRM** for sales — contacts, deals, pipeline, activities
- **Accounting** for finance — invoices, payments, revenue, expenses
- **Billing / Subscription** for recurring revenue — subscriptions, charges, churn events
- **HRIS** for people — employees, departments, compensation, time off
- **Support** for customers — tickets, satisfaction scores, resolution times
- **Project management, marketing automation, and others** depending on the business

Each of these systems works within its silo. The problem is that **business questions don't live within silos**. The questions that matter most — customer health, team capacity, revenue trends, churn risk — require data from multiple systems simultaneously.

The Problem: Cross-System Blindness

Cross-system blindness is the inability to get a unified, reliable, real-time view of the business across all connected systems. It's not that the data doesn't exist — it does. And it's not always that there's zero reporting — some companies have partial answers, cobbled together manually or through expensive pipelines. The problem is that the answers are either missing entirely, unreliable, or prohibitively expensive to maintain.

It shows up in questions that no single tool can answer reliably:

- **"What is our customer health across the base?"** — requires CRM engagement + support satisfaction + billing status + payment history. A customer can look "healthy" in the CRM (active deals, recent meetings) while silently accumulating overdue invoices in accounting and unresolved escalations in support. Some companies approximate this in a spreadsheet, but it's a snapshot that's stale by the time it's shared, and nobody knows which numbers to trust.
- **"Is our sales team at capacity or do we need to hire?"** — requires pipeline load + headcount + PTO schedules + quota attainment. The CRM shows pipeline, the HRIS shows headcount, but the VP of Sales ends up eyeballing both screens side by side. Companies with a data team can build this report, but it breaks every time someone changes the CRM pipeline stages or the HRIS adds a new employment type.
- **"Which customers are at risk of churning this quarter?"** — requires declining product engagement + open support escalations + late payments + contract renewal dates. Each signal lives in a different system. A few companies have rigged up Zapier workflows or scheduled exports to flag churn signals, but these are fragile — when they break, nobody notices until a customer has already left.
- **"What is our actual revenue per employee, and how does it trend?"** — requires accounting revenue + HRIS headcount over time. Simple in concept, but impossible to compute automatically without linking the two systems and maintaining a historical record of both. Even companies that compute this manually can't tell you whether the number went up because revenue grew or because headcount shrank.

- **"How does our pipeline-to-revenue conversion compare to last quarter?"** — requires CRM pipeline data + accounting closed revenue + the ability to match which deal became which invoice. This is an entity matching problem: the CRM calls it a "Deal," accounting calls it an "Invoice," and the connection between them exists only in someone's head or in a manually maintained lookup.

The data exists. The questions are well-understood. What's missing is the infrastructure to answer them reliably, automatically, and affordably — with confidence that the numbers are correct.

The Three Scenarios: It's Not Just "No Reporting"

Cross-system blindness is not a binary problem. Companies don't simply "have" or "lack" cross-system reporting. In practice, SMBs fall into one of three scenarios — and each has a distinct pain point, but all share the same underlying needs.

Scenario 1: No cross-system reporting exists.

The company has good individual tools but no way to connect them. Cross-system questions are answered by gut feel, anecdote, or not at all. The CEO knows revenue is growing but can't say which customers are healthy and which are silently deteriorating. The VP of Sales knows the pipeline looks thin but can't quantify whether the team is under-staffed or under-performing.

These companies want cross-system visibility but have been priced out of every solution that could deliver it. A data warehouse project costs $125K-$335K. A full-time data engineer costs $120K+/year. Enterprise BI tools require both. **They need an affordable, turnkey solution that doesn't require a data team.**

Scenario 2: Cross-system reporting exists, but it's manual and brittle.

This is the most common scenario. Someone — usually the CEO, CFO, or VP of Operations — has built a "Monday spreadsheet": a Google Sheet that pulls numbers from four or five systems, pastes them together, and uses formulas to approximate cross-system metrics.

- **Time cost:** 3-8 hours per week to maintain
- **Reliability:** Error-prone — manual copy-paste with no validation, formulas that break when source data changes
- **Staleness:** The output is outdated the moment it's saved
- **Accessibility:** Only the person who built the spreadsheet understands how it works
- **Continuity:** Breaks when that person goes on vacation, changes roles, or leaves the company
- **No data quality visibility:** Nobody knows if the numbers are right, incomplete, or double-counted

Some companies have gone a step further: automated parts of this with Zapier workflows, scheduled exports, or basic ETL scripts. These work until they don't. The pipelines are fragile, undocumented, and nobody owns them. When a SaaS vendor changes their API, the pipeline breaks silently and the numbers drift. **They need reliability, data quality assurance, and a system that doesn't depend on one person's tribal knowledge.**

Scenario 3: Cross-system reporting is fully automated, but expensive to maintain.

A small number of SMBs have invested in a proper data stack: a cloud warehouse (Snowflake, BigQuery), ETL pipelines (Fivetran, Stitch), a BI tool (Looker, Tableau), and 1-3 data engineers to build and maintain it.

This works — but it's expensive, fragile in a different way, and still incomplete:

- **Cost:** $200K-$400K/year fully loaded (tooling + headcount)
- **Maintenance burden:** Every new SaaS tool, every API change, every new metric requires engineering work
- **No entity resolution:** Most SMB data stacks dump data into a warehouse but don't solve the identity problem — the same customer is still "Acme Corp" in HubSpot and "ACME LLC" in QuickBooks with no automatic link
- **No proactive intelligence:** They have dashboards, but dashboards are reactive — someone has to look at them to notice something has changed
- **Data quality is a constant battle:** Pipeline failures, stale syncs, schema changes, and duplicate records erode trust

They need to reduce the cost and complexity of maintaining their data stack while adding the entity resolution, data quality monitoring, and proactive AI intelligence their current setup lacks.

The Common Thread: What Every Scenario Needs

Across all three scenarios, the same four needs emerge:

1. **A unified entity view across systems.** The same customer, employee, or deal appears in multiple systems under different names, IDs, and schemas. Without resolving these identities, cross-system metrics are impossible. Entity resolution is the foundational capability that makes everything else work — and it's the capability that no current SMB tool provides.
2. **Reliable, governed cross-system metrics with data quality assurance.** Whether the reporting is manual, automated, or nonexistent, data quality is the universal problem. Numbers that can't be trusted are worse than no numbers at all. Every company needs to know not just "what are the metrics" but "how reliable are the metrics" — with confidence scores, freshness indicators, and clear explanations of what's missing or incomplete.
3. **Agentic analytics — proactive intelligence, not reactive dashboards.** Dashboards require someone to look at them. The CEO of a 200-person company doesn't have an analytics team monitoring dashboards. They need intelligence that comes to them: weekly briefings that synthesize the full picture, alerts when cross-system signals converge on a problem, and the ability to ask a question and get a reliable answer from all systems at once.
4. **An affordable, managed data warehouse that doesn't require a data team.** The warehouse is the foundation — decoupling analytics from operational systems, storing historical data, enabling joins across sources. But the warehouse itself is not the product. It's the invisible infrastructure that makes entity resolution, governed metrics, and AI intelligence possible. Companies that have no warehouse need one provisioned automatically. Companies that already have one need the intelligence layers on top.

What Needs to Exist

The solution requires five capabilities working together — no subset is sufficient:

1. **A managed data warehouse** that provisions itself when the customer connects their tools — invisible, zero-configuration, zero-maintenance.
2. **Cross-system entity resolution** that automatically identifies when the same customer, employee, or deal appears across multiple systems and links those records together — without requiring a data engineer or an MDM tool.
3. **A certified semantic layer** with pre-built, governed metric definitions that span multiple systems — so that "customer health" and "revenue per employee" are computed once, defined precisely, and available everywhere.
4. **A data quality engine** that continuously monitors the reliability of every data source, propagates confidence scores through the pipeline, and explains the business impact of any data gap — so the customer can trust every number the system reports.
5. **A proactive AI analyst** that synthesizes all of the above into weekly briefings, exception alerts, and on-demand answers — grounded in certified metrics, not hallucinated from raw data.

This is the problem DataOrbit AI exists to solve. Not a better dashboard. Not a chatbot. A complete, AI-first data intelligence platform that brings a unified view of data, reporting, and agentic analytics to companies that have been priced out of every existing solution — and that reduces cost and complexity for companies that have already invested in a data stack but aren't getting the intelligence they need from it.

1.2 Why Now

Three market developments make this the right time to build this product:

The semantic layer has matured. dbt open-sourced MetricFlow under Apache 2.0 in October 2025. Cube Core is a production-grade open-source semantic layer. The industry has converged on how to define, govern, and serve certified metrics. DataOrbit doesn't need to invent this technology — it can embed it.

AI agents can now generate reliable narrative intelligence. Large language models can synthesize structured data into clear, actionable business narratives — but only when grounded in governed data with certified metric definitions. Without a semantic layer, AI hallucinates metrics. With one, it becomes a trustworthy analyst.

Enterprise agentic analytics is proving the concept, but excluding SMBs. ThoughtSpot (Gartner Leader in BI, 2025) launched its "Agentic Analytics Platform." Cube evolved into an agentic analytics company. Tableau integrated with Salesforce's Agentforce. The concept of AI agents that understand business data is being validated at the enterprise level — but every one of these products requires a pre-existing data warehouse, a data engineering team, and enterprise budgets. The 50-500 employee company with no data team is locked out.

DataOrbit brings agentic analytics to the companies that need it most and can afford it least to build themselves.

1.3 The Company

DataOrbit AI is the company. The product is an **agentic data warehouse**: a platform that provisions a

managed data warehouse when the customer connects their SaaS tools, applies pre-built entity matching and metric definitions on top of it, and delivers proactive AI intelligence — all without requiring the customer to build, configure, or maintain anything.

DataOrbit AI supports AI integrations from the very beginning of setting up a warehouse through to delivering agentic analytics. The warehouse isn't a separate step the customer has to take before getting intelligence — it's the invisible foundation that makes intelligence possible.

1.4 The Positioning Statement

> **DataOrbit AI is the first agentic data warehouse built for companies that don't have a data team.**
> It's the entire data stack in one product: a managed warehouse that provisions itself when you connect your tools, a semantic layer with certified cross-system metrics pre-built for your industry, an entity matching engine that unifies customers, employees, and deals across every system, and a proactive AI analyst that delivers intelligence before you know to ask.
> What normally takes a data team, a warehouse project, a BI tool, and 6-12 months of setup — DataOrbit delivers in days.

PART 2: THE MARKET

2.1 Target Customer Profile

Primary ICP: The "Systemized but Siloed" SMB

Company size: 50-500 employees **Revenue:** $5M-$100M **Data maturity:** Low to medium — no data warehouse, no data engineering team, no dedicated analytics function **System maturity:** Medium to high — they run standardized SaaS tools for CRM, ERP, HRIS, and support

Key characteristics:

These companies have invested in good tools for each department. Sales uses HubSpot or Salesforce. Finance uses QuickBooks or Xero. HR uses BambooHR or Gusto. Support uses Zendesk or Intercom. Each tool works well within its silo. The company has graduated from "doing everything in spreadsheets" to "each department has its own system."

But they've hit a wall. The CEO can't get a single view of the business without pulling data from four systems into a spreadsheet. The CFO can't compute revenue per employee without combining QuickBooks and BambooHR data manually. The VP of Sales can't assess rep capacity without cross-referencing the CRM pipeline with HRIS headcount data. Nobody can compute "customer health" because it requires CRM engagement + support satisfaction + billing status — three different systems.

They've looked at BI tools (Tableau, Power BI, Looker Studio) and found they need a data warehouse, an ETL pipeline, and someone who knows SQL to make them useful. They've tried AI analytics tools (ChatGPT with uploaded CSVs, Copilot) and lost trust when the answers were wrong. They've considered hiring a data engineer but can't justify a $120K+ salary for a single-person function. So they stay stuck in the Monday spreadsheet ritual.

What they want:

- A Monday morning view of the entire business without manual effort
- Alerts when something unusual happens — before it becomes a crisis
- The ability to ask a question and get a reliable answer from all their systems at once
- Cross-system reports they can share with their board, leadership team, or investors

What they don't want:

- A 6-month implementation project
- To hire a data engineer
- To learn SQL, YAML, or any technical language
- Another dashboard they have to remember to check

Secondary ICP: The "Outgrowing Spreadsheets" Scale-Up

Company size: 100-1,000 employees

These companies may have a small analytics or ops team (1-3 people), possibly a basic warehouse (BigQuery, a Postgres replica), and a BI tool (Metabase, Looker Studio). They want the governed cross-system metric layer and AI agent to augment their existing stack — not replace it.

For these customers, DataOrbit's semantic layer and AI agent become the intelligence layer on top of their existing infrastructure, delivered through Cube's SQL API to their existing BI tools.

2.2 Target Verticals

We are launching with **B2B SaaS** as the first vertical. This choice is deliberate:

Vertical	Why First	Primary System Combination
B2B SaaS	Subscription lifecycle is data-rich; CRM + billing + support is standard; founders/CEOs are tech-savvy enough to try a new tool quickly	HubSpot + Stripe + QuickBooks (+ optionally Zendesk, BambooHR)

After establishing product-market fit in B2B SaaS, we expand to:

Vertical	System Combination	Key Cross-System Questions
Professional Services	HubSpot + Harvest/Toggl + QuickBooks + BambooHR	Project profitability, utilization vs. revenue, team capacity
E-commerce / DTC	Shopify + Klaviyo + Gorgias + QuickBooks	Customer LTV by channel, return rate impact, inventory turnover
Healthcare Clinics	Athenahealth + QuickBooks + Gusto + Zendesk	Patient revenue per provider, scheduling utilization, compliance
Financial Services (SMB)	Wealthbox + QuickBooks + BambooHR	AUM per advisor, client profitability, regulatory metrics
Construction	Procore + QuickBooks + Gusto	Project profitability, labor cost vs. estimate, materials variance

Each vertical has a specific template: pre-built entity matching rules, metric definitions, derived entity states, and briefing formats tailored to that industry and system combination.

2.3 The Competitive Landscape

After comprehensive competitive research across 20+ products, we've identified five competitive tiers. No competitor occupies the specific position DataOrbit targets: a complete, pre-configured data stack (warehouse + semantic layer + entity matching + AI agent) for SMBs without data teams.

Tier 1: Enterprise Cloud Data Warehouses (Snowflake, BigQuery, Databricks)

Not competitors — different buyer entirely. These serve data engineering teams at 1,000+ employee companies with $500K+ data budgets. They are infrastructure, not intelligence. DataOrbit's managed warehouse serves the same functional purpose (decoupling analytics from operational systems) but is invisible to the customer.

Relationship to DataOrbit: DataOrbit is what these platforms do, delivered as one product that sets itself up. Potential future migration path for Scale-tier customers who outgrow DataOrbit.

Tier 2: Managed SMB Warehouses (Panoply, Mozart Data, DataSelf ETL+)

The closest competitors by target buyer. These simplify data warehousing for smaller companies. Panoply offers managed BigQuery with connectors. Mozart bundles Snowflake + Fivetran. DataSelf offers pre-built KPI libraries for ERP users.

Where they fall short: They deliver warehouse infrastructure, not business intelligence. You get a database with your data in it — now go find a BI tool, write SQL, and figure out your own metrics. No entity matching, no proactive AI, no governed semantic layer, no pre-built intelligence.

Our positioning vs. this tier: "They give you a warehouse. We give you a warehouse that already understands your business."

Tier 3: Enterprise Agentic Analytics (ThoughtSpot, Tableau Next, Cube.dev, Tellius)

This tier is defining the "agentic analytics" category. ThoughtSpot is a 2025 Gartner Leader. Cube's semantic layer powers agentic AI queries. Tableau integrates with Salesforce Agentforce.

Where they fall short for our ICP: Every one requires a pre-existing data warehouse (BYO Snowflake, BigQuery, etc.). Every one requires a data team to configure. Pricing starts at ~$2,750/month for ThoughtSpot. They're enterprise tools for enterprise buyers.

Our positioning vs. this tier: "They add intelligence to your data stack. We are the data stack — warehouse, semantic layer, and intelligence — in one product."

Tier 4: SMB Dashboard & KPI Tools (Databox, Klipfolio, Narrative BI)

What our ICP currently uses. These connect to SaaS tools and display metrics in dashboards. Databox offers 4,000+ pre-defined metrics across 100+ sources. Narrative BI generates proactive AI insights and is recognized in Gartner's 2025 Market Guide for Agentic Analytics.

Where they fall short: No warehouse layer for persistent historical data. No cross-system entity matching. No cross-system metrics (they show per-system metrics side by side). No derived entity states. Narrative BI is the closest in philosophy but operates within single-system data boundaries.

Our positioning vs. this tier: "They show you numbers from each tool. We tell you what those numbers mean together."

Narrative BI is the most dangerous competitor — closest in philosophy and Gartner-recognized. We must establish the cross-system intelligence category before they add a warehouse layer and entity matching (estimated 12-18 months of development for them).

Tier 5: AI Chatbots (ChatGPT, Copilot)

What our ICP has tried and lost trust in. No persistent data integration, no business context, high hallucination risk, single-system only.

Our positioning vs. this tier: "We're not a chatbot that guesses. We're a governed intelligence system built on a real warehouse, with certified metrics that can't be hallucinated."

The True Competitor: Spreadsheets

The CEO pulling numbers from four systems into a Google Sheet every Monday morning is the actual behavior we're displacing. This ritual is hated but deeply entrenched. It's free, familiar, and "works" (poorly). Every marketing message should make the spreadsheet ritual feel absurd compared to what DataOrbit delivers.

2.4 Three Defensible Differentiators

Differentiator 1: The Self-Provisioning Intelligent Warehouse

DataOrbit includes a managed data warehouse as part of the product. When the customer connects their SaaS tools via OAuth, DataOrbit automatically provisions the warehouse, extracts data, normalizes it, and keeps it synced. The customer never sees, configures, or maintains the warehouse. It exists as the invisible foundation for everything above it.

This is not "no warehouse" — that would mean querying operational systems directly, which is neither practical nor responsible. The differentiator is that the warehouse is **built in and invisible**. The customer skips the warehouse *project*, not the warehouse itself.

Differentiator 2: Pre-Built Cross-System Intelligence Per System Combination

When a customer connects HubSpot + Stripe + QuickBooks, they don't get a blank warehouse with raw tables. They get a working intelligence layer: entity matching rules that link customers across all three systems, 30+ certified cross-system metrics, derived entity states, and alert thresholds — all pre-configured for that specific system combination.

Nobody else ships this. DataSelf has KPI libraries but ERP-only. Databox has 4,000 metrics but single-system. No one delivers pre-built, industry-specific, cross-system intelligence templates.

Differentiator 3: Proactive Cross-System AI Analyst

Monday briefings that synthesize the entire business. Exception alerts when cross-system signals converge. On-demand answers grounded in certified metrics. All warehouse-backed, all reasoning across every connected system simultaneously. Not reactive dashboards. Not single-metric alerts. Proactive, cross-system, narrative intelligence.

PART 3: THE PRODUCT

3.1 The Customer Experience

What the Customer Sees

DataOrbit presents itself as an AI analyst that knows the customer's entire business. The customer's experience is:

1. **Connect** — Authenticate their SaaS tools via OAuth (HubSpot, Stripe, QuickBooks, etc.)
2. **Confirm** — Review and confirm entity matches that DataOrbit isn't fully confident about ("We think Acme Corp in HubSpot is the same as ACME LLC in QuickBooks — can you confirm?")
3. **Customize** — Review the pre-built metric definitions and adjust if needed ("Yes, this is how we define revenue" / "No, we exclude refunds from this metric")
4. **Receive** — Start receiving weekly briefings, exception alerts, and answer questions via chat

The customer never sees the warehouse. They never write SQL. They never configure a data pipeline. They never build a dashboard (unless they want to, in which case they can connect their BI tool to DataOrbit's SQL API).

What the Customer Does Not See

Behind the scenes, DataOrbit:

- Provisions a managed Postgres warehouse

- Extracts data from each connected system via API
- Stores it in system-specific schemas (hubspot.companies, stripe.customers, etc.)
- Normalizes raw data into a canonical data model (system-agnostic)
- Profiles data quality: completeness, freshness, volume consistency, schema stability
- Runs entity matching to link records across systems
- Builds a unified entity graph with confidence scores on each match
- Applies the semantic layer (Cube Core) with pre-built metric definitions
- Computes derived entity states (customer health, employee capacity, deal risk)
- Assigns confidence scores to every metric based on upstream data quality
- Generates AI briefings by querying the semantic layer and summarizing via LLM
- Monitors metrics for threshold violations and sends alerts

All of this is infrastructure. The customer's mental model is simply: "I connected my tools and DataOrbit knows my business."

3.2 The Six-Layer Architecture

DataOrbit's product architecture has six layers, each building on the one below it:

```
┌──────────────────────────────────────────────────────────────────────┐
│ Layer 6: AI Agent (DataOrbit Apollo)                                 │
│ Briefings • Alerts • Q&A — all grounded in certified metrics         │
├──────────────────────────────────────────────────────────────────────┤
│ Layer 5: Semantic Layer (Cube Core)                                  │
│ Certified metric definitions • SQL generation • Caching • APIs       │
├──────────────────────────────────────────────────────────────────────┤
│ Layer 4: Unified Entity Graph                                        │
│ Cross-system entity resolution • Derived states • AI-readable context│
├──────────────────────────────────────────────────────────────────────┤
│ Layer 3: Entity Matching Engine                                      │
│ Deterministic → Probabilistic → Human confirmation                   │
├──────────────────────────────────────────────────────────────────────┤
│ Layer 2: Data Quality Engine                                         │
│ Source profiling • Anomaly detection • Impact analysis • Confidence  │
├──────────────────────────────────────────────────────────────────────┤
│ Layer 1: Managed Warehouse (Postgres) + Data Connectors              │
│ OAuth ingestion • Schema normalization • Scheduled sync              │
└──────────────────────────────────────────────────────────────────────┘
```

Each layer is described in detail below.

Layer 1: Managed Warehouse + Data Connectors

What it does: Connects to the customer's SaaS systems, extracts relevant data on a scheduled basis, and stores

it in a managed Postgres warehouse. The customer never sees, provisions, or maintains this warehouse — it's an implementation detail.

Data ingestion approach:

Category	Systems (MVP)	Data Extracted
CRM	HubSpot	Contacts, companies, deals, activities, lifecycle stages, pipeline data
Billing / Subscription	Stripe	Customers, subscriptions, invoices, charges, MRR, churn events, payment failures
Accounting	QuickBooks Online	Customers, invoices, payments, expenses, chart of accounts, P&L
Support (Growth tier)	Zendesk	Tickets, satisfaction scores, response times, resolution times
HRIS (Growth tier)	BambooHR	Employees, departments, compensation, time off, headcount changes

Refresh cadence:

- Default: Daily sync (overnight batch) for most data
- Hourly: Critical event data — new deals closed, tickets escalated, churn events, payment failures
- On-demand: Triggered when the user asks a question requiring fresh data

Technology choice: Postgres (managed — Supabase, Neon, or AWS RDS). SMB data volumes (thousands to tens of thousands of records per system) don't require Snowflake or BigQuery. Postgres is simple, reliable, Cube-compatible, and cost-effective.

Data ingestion technology: Airbyte (open-source, self-hostable) for pre-built connectors, or custom REST API extraction for the MVP's three core systems (HubSpot, Stripe, QuickBooks). Airbyte offers 300+ pre-built connectors and handles OAuth, pagination, rate limiting, schema detection, and incremental sync. Custom extraction gives more control but requires more engineering.

Internal schema design:

Data flows through three schema layers, each serving a specific purpose:

```
raw.*          → System-specific tables, preserving source structure exactly
canonical.*    → Normalized tables, system-agnostic (the canonical data model)
analytics.*    → Unified tables joined via entity_links, ready for the semantic layer
```

Layer 1: Raw tables land in system-specific schemas that preserve the source system's structure:

```
raw.hubspot_companies
raw.hubspot_contacts
raw.hubspot_deals
raw.stripe_customers
raw.stripe_subscriptions
raw.stripe_invoices
raw.quickbooks_customers
raw.quickbooks_invoices
raw.quickbooks_payments
```

Layer 2: Canonical tables normalize raw data into system-agnostic representations. Each connector module maps its system's fields to the canonical model (described in detail in Section 4.4, Template Architecture):

```
canonical.companies      (from hubspot.companies, quickbooks.customers, stripe.customers)
canonical.contacts       (from hubspot.contacts, stripe.customers)
canonical.deals          (from hubspot.deals)
canonical.invoices       (from quickbooks.invoices, stripe.invoices)
canonical.subscriptions  (from stripe.subscriptions)
```

Each canonical table includes `source_system` and `source_system_id` fields, enabling the entity matching engine to operate on normalized data without system-specific logic.

Layer 3: Analytics tables are the output of the transformation layer, which joins canonical tables via `entity_links` into unified, cross-system views:

```
analytics.companies      (unified from all systems via entity_links)
analytics.contacts       (unified from all systems via entity_links)
analytics.deals          (from CRM, enriched with billing data)
analytics.invoices       (unified from accounting + billing)
analytics.subscriptions  (from billing)
analytics.entity_states  (derived from cross-system rules)
```

The analytics tables are what the semantic layer (Cube Core) queries. The canonical tables are what the entity matching engine and data quality engine operate on. The raw tables are what the connectors write to and what schema stability monitoring tracks.

Layer 2: Data Quality Engine

What it does: Continuously monitors the health, completeness, freshness, and consistency of data flowing through the pipeline — from raw ingestion through to AI agent output. Acts as the immune system of the data pipeline: a primary checkpoint between raw data and entity matching, with monitoring sensors at every downstream boundary.

Why this layer exists:

Every layer in the architecture trusts the layer before it. Entity matching trusts that raw data has usable fields. The semantic layer trusts that entity matching produced correct links. The AI agent trusts that metrics are accurate. If source data is garbage — missing fields, duplicates, stale records, schema changes — the garbage propagates silently through every downstream layer.

One wrong number in one Monday briefing destroys customer trust permanently. The Data Quality Engine prevents this by catching problems before they propagate and communicating the reliability of every insight to the customer.

Where it operates:

```
SaaS APIs → Raw Tables → [PRIMARY QUALITY GATE] → Entity Matching → Entity Graph → Semantic Layer →
AI Agent
                ↑                ↑          ↑          ↑
            Source quality     Match quality  Metric quality  Output quality
            (heaviest checks)   monitoring    monitoring     monitoring
```

The primary gate runs after every data sync, before data moves to entity matching. Quality monitors at each subsequent boundary track how quality issues propagate downstream.

Quality Check Category 1: Source Data Quality (after ingestion, before entity matching)

This is the primary gate — the most thorough set of checks.

Completeness profiling: After each sync, the engine measures the population rate of every field that matters downstream. "HubSpot companies: name populated 99.2%, domain populated 76.4%, industry populated 41.8%, owner populated 88.1%." This is not pass/fail — it is a profile that propagates to downstream layers. Entity matching reads this profile and adjusts its reliance on each field (e.g., if domain is only 76% populated, weight domain matching lower). The AI agent reads it to qualify its outputs: "Customer health scores are computed for 76% of your customer base — the remaining 24% are missing domain information needed for cross-system matching."

Freshness monitoring: "Last successful HubSpot sync: 4 hours ago. Last successful QuickBooks sync: 27 hours ago (expected: daily)." If a sync is stale, every metric derived from that system is stale. The engine flags this prominently throughout the pipeline — every downstream metric, alert, and briefing section that depends on stale data is annotated with the staleness.

Volume consistency: "HubSpot companies: 347 records yesterday, 344 today (3 deleted — normal). QuickBooks customers: 289 records yesterday, 14 today (275 missing — ALERT: possible API pagination failure or bulk delete)." A sudden drop in record count almost always indicates an ingestion failure, not a real business event. Catching this before it propagates prevents the AI agent from reporting "revenue dropped 95%" when the data pipeline broke.

Schema stability: "HubSpot: new property detected — `hs_v2_deal_amount` (number). Existing property `deal_amount` may be deprecated." SaaS vendors change their APIs. If DataOrbit doesn't detect this, the connector module maps to a field that's no longer populated, and metrics silently go to zero.

Duplicate detection: "QuickBooks: 4 customer records with identical names and email addresses detected. These may be duplicates in your QuickBooks data. If they are, your revenue metrics will double-count their

invoices." DataOrbit cannot fix duplicates in the source system, but it can detect them and warn the customer before the duplicates corrupt downstream metrics.

Value validity: "Stripe subscription amount: $-500 detected for customer XYZ. Negative subscription amounts are unusual — this may be a refund miscategorized as a subscription." Range checks, type checks, and format checks catch data entry errors and system bugs before they flow into metrics.

Quality Check Category 2: Entity Matching Quality (after matching, before entity graph)

Match coverage: "85% of HubSpot companies are matched to QuickBooks customers. 12% are under review. 3% have no match candidates." This tells the customer and the AI agent how complete the cross-system picture is.

Match confidence distribution: "Of 294 matched entities: 231 high confidence (deterministic match), 48 medium confidence (probabilistic), 15 low confidence (awaiting confirmation)." Metrics computed from low-confidence matches should carry different reliability indicators than metrics from high-confidence matches.

Match conflict detection: "Entity Acme Corp has conflicting attributes: HubSpot says San Francisco, QuickBooks says New York. This conflict affects the accuracy of any location-based analysis."

Orphan analysis: "47 QuickBooks customers have no match in HubSpot. These represent $127K in monthly revenue that is not connected to any CRM pipeline data. Your 'Pipeline to Revenue Conversion' metric is computed only on matched entities — this $127K is excluded." This tells the customer exactly what they're missing and why — something no standalone data quality tool can do without knowledge of the downstream metric definitions.

Quality Check Category 3: Metric Quality (after semantic layer computation)

Cross-system reconciliation: "Monthly revenue from QuickBooks paid invoices: $487K. Monthly revenue from Stripe successful charges: $412K. Difference: $75K. Likely explanation: professional services revenue exists in QuickBooks but not in Stripe." This runs as an automated recurring check, not a one-time onboarding step.

Metric stability monitoring: "Revenue Per Employee changed by 34% this week. Historical volatility: typically <5% week over week. This is 6.8 standard deviations from normal. Checking drivers: headcount changed (4 employees terminated this week) while revenue was unchanged. Root cause: headcount change, not revenue anomaly."

Null propagation tracking: "Customer Health Score: computed for 248 of 294 matched entities (84%). Missing scores: 31 missing support data (Zendesk not connected for these), 12 missing engagement data (no HubSpot activity in 90 days), 3 missing financial data (no QuickBooks invoices found)."

Quality Check Category 4: Output Quality (what the AI agent reports)

Confidence tagging on every AI output. Every number the AI agent reports carries a quality indicator derived from the upstream quality checks:

High confidence example: "Your MRR is **$412,000** *(high confidence — based on Stripe data synced 2 hours ago, 100% of subscriptions captured, no anomalies detected)*."

Medium confidence example: "Your Customer Health Score average is **72** *(medium confidence — computed for 84% of your customer base; 16% of customers are excluded due to incomplete cross-system data. The 84% that are scored have high-quality data from all connected systems)*."

Low confidence example: "Revenue Per Employee is **$3,420** *(low confidence — BambooHR sync is 3 days stale due to an API authentication failure; headcount may not reflect recent changes. Revenue data from QuickBooks is current. We recommend re-authenticating BambooHR before using this figure for decisions)*."

Confidence is not a single flag — it is computed from the upstream quality signals: source freshness, field completeness, match confidence, and metric stability. The engine combines these signals into a composite confidence score that propagates to every consumer — the AI agent, the dashboards, and the API.

Downstream Impact Analysis — the genuinely novel capability:

Because DataOrbit owns the entire pipeline from raw data to AI output, the Data Quality Engine can do something no standalone data quality tool can: trace the business impact of any quality issue through every downstream layer.

Example of a complete downstream impact trace:

"**Data quality issue:** HubSpot `companies.domain` is populated for only 76% of records.

Impact on entity matching (Layer 3): Domain matching is one of three matching signals for CRM ↔ Accounting linkage. With 24% of companies missing domain, entity match coverage is estimated at 78% instead of the potential 89% if domain were fully populated. Approximately 32 companies in QuickBooks are not linked to their HubSpot records.

Impact on entity graph (Layer 4): These 32 unlinked companies exist in QuickBooks but have no corresponding entity in the unified graph. They have no derived state (Healthy, At Risk, etc.) because state derivation requires cross-system signals.

Impact on metrics (Layer 5): The 32 unlinked companies represent an estimated $47K in monthly revenue. This revenue IS counted in Total Revenue (which comes from QuickBooks alone) but is NOT included in cross-system metrics: Customer Health Score, Pipeline-to-Revenue Conversion, or Revenue Per Rep. These metrics are accurate for the 78% that are matched but exclude the unlinked $47K.

Impact on AI agent (Layer 6): The Monday briefing's At-Risk customer list is computed only for matched entities. If any of the 32 unlinked companies are showing signs of churn in QuickBooks (late payments, declining invoices), the AI agent cannot detect it because it doesn't know which HubSpot company they correspond to. The AI agent's briefing will include a note: 'Customer health monitoring covers 78% of your revenue by customer count. 32 customers representing $47K/month are not yet linked across systems.'

Recommended action: Populate the domain field for the 83 HubSpot companies missing it. DataOrbit can export this list. Alternatively, we can attempt to infer domains from contact email addresses — 18 of the 83 companies have contacts with business email addresses (e.g., jane@acme.com → acme.com). This would increase match coverage to an estimated 84% and bring an additional $28K under cross-system monitoring."

This impact trace is generated by AI using the Data Quality Engine's observations, the entity matching rules, the metric definitions, and the influence maps. It requires knowledge of the entire pipeline — which is exactly what DataOrbit has.

What the customer sees — three views:

Pipeline Health View: A single screen showing the health of the entire data pipeline from source systems to AI output. Each layer has a status indicator (green/yellow/red) with drill-down. The CEO sees at a glance: "All systems syncing, entity matching at 85%, all metrics computing, briefing is reliable."

System Health View: Per-system detail. "HubSpot: 347 companies, 1,204 contacts, 89 deals. Last sync: 2 hours ago. Field completeness: name 99%, domain 76%, industry 42%. Data freshness: 15 records updated since last sync. Issues: 4 potential duplicates detected."

Impact View: Quality issues ranked by downstream business impact, not technical severity. "Issue #1: 83 companies missing domain — estimated $47K revenue unmonitored. Issue #2: QuickBooks sync 27 hours stale — all financial metrics affected. Issue #3: 4 duplicate customer records — estimated $12K revenue double-counted."

Layer 3: Entity Matching Engine

What it does: Identifies when the same real-world entity (company, contact, employee) appears across multiple connected systems and links those records together.

Why this is the hardest technical problem in the platform:

SaaS systems use fundamentally different entity models. HubSpot identifies companies by domain name. QuickBooks identifies customers by display name (which can be a company or a person — there's no structural distinction). Stripe identifies customers by email. There is no universal key.

The matching signals vary by system pair:

Signal	HubSpot → QuickBooks	HubSpot → Stripe	HubSpot → BambooHR
Email	Medium (different contacts may exist)	High (billing email often present)	High (work email matches)
Domain/website	Medium (HubSpot has it; QuickBooks usually doesn't)	Low (Stripe rarely stores domain)	N/A
Company name	Present in both, but varies ("Acme Corp" vs "ACME LLC")	Partially (Stripe metadata)	Present (employer name)
External ID	If HubSpot stores QB/Stripe ID as custom property	If HubSpot stores Stripe customer ID	N/A

Critical design constraint: Precision over recall. A false positive (incorrectly merging two different companies) is far more dangerous than a false negative (failing to link two records for the same company). If DataOrbit says "Acme Corp's revenue dropped 40%" because it conflated two companies, trust is destroyed permanently. We target >95% precision, accepting lower recall that improves over time.

Tiered matching approach:

Tier 1 — Deterministic Matching (MVP):

Match on high-confidence identifiers after normalization:

- **Email match:** Normalize (lowercase, trim) and match exactly. If the billing email in Stripe matches a contact's email in HubSpot, and that contact is associated with a company — link the Stripe customer to the HubSpot company.
- **Domain match:** Extract domain from email addresses (user@acme.com → acme.com), normalize (strip www, lowercase), match against HubSpot's Company Domain field.
- **External ID match:** If HubSpot stores a Stripe customer ID or QuickBooks customer ID as a custom property (common in companies using integration tools like Zapier), match directly.
- **Exact name match after normalization:** Lowercase, strip legal suffixes (Inc, LLC, Corp, Ltd, AG, GmbH), strip punctuation, trim whitespace. Match exact normalized names.

Expected coverage: 60-70% of entities matched with high confidence.

Tier 2 — Probabilistic Matching (v2, post-MVP):

For records that don't match deterministically, apply probabilistic matching using Splink (open-source, Python, DuckDB backend). Splink implements the Fellegi-Sunter model — it computes match weights for each comparison field (name similarity, address similarity, phone similarity) and produces a probabilistic match score.

Key properties of Splink for our use case:

- Unsupervised — doesn't require labeled training data from the customer
- Fast — can link a million records on a laptop in under a minute (our volumes are orders of magnitude smaller)
- Production-proven — used by NHS England, UK Ministry of Justice, UK Ministry of Defence
- DuckDB backend — lightweight, embeddable, no infrastructure required

Configuration: Set threshold for auto-merge (high confidence), auto-reject (low confidence), and human review (middle band). DataOrbit pre-configures thresholds per system pair based on learnings from previous deployments.

Expected additional coverage: 15-25% of remaining unmatched entities, with some requiring human confirmation.

Tier 3 — Human-in-the-Loop:

Present ambiguous matches to the customer during onboarding: "We think Acme Corp in HubSpot is the same as ACME LLC in QuickBooks — can you confirm?" This is the safety net that maintains precision. Build a simple review interface showing the candidate match, the signals that suggested it, and a confidence score. The customer clicks "Yes, same entity" or "No, different entities."

Over time, confirmed and rejected matches become training data that improves matching for similar entities in this customer's data and (in aggregate, anonymized) for the matching templates used by other customers on the

same system combination.

Tier 4 — LLM-Assisted (Future):

For mature deployments with remaining edge cases, use an LLM to evaluate candidate pairs. The LLM receives the records from both systems and determines whether they describe the same entity, considering context (industry, location, size) that rule-based systems miss.

Current LLM accuracy on entity matching tasks is estimated at approximately 80-85% based on general benchmarks and internal testing (this figure should be validated with DataOrbit's specific entity types and system combinations before production deployment). This is useful as a second-pass on uncertain candidates but too expensive and slow for bulk matching.

Entity match output:

The entity matching engine produces two tables — an extensible design that scales to any number of connected systems without schema changes:

```sql
-- Each real-world entity gets one row
CREATE TABLE entities (
    entity_id UUID PRIMARY KEY,
    entity_type VARCHAR(50),        -- 'company', 'contact', 'employee'
    display_name VARCHAR(255),      -- Best available name for this entity
    overall_confidence VARCHAR(20),  -- 'high', 'medium', 'low' (lowest of all matches)
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);

-- Each source system record linked to an entity gets one row
CREATE TABLE entity_links (
    link_id UUID PRIMARY KEY,
    entity_id UUID REFERENCES entities(entity_id),
    source_system VARCHAR(50),      -- 'hubspot', 'quickbooks', 'stripe', etc.
    source_entity_type VARCHAR(50),  -- 'company', 'customer', 'account', etc.
    source_record_id VARCHAR(100),   -- ID in the source system
    match_confidence VARCHAR(20),    -- 'high', 'medium', 'low'
    match_method VARCHAR(50),       -- 'email', 'domain', 'name_exact', 'probabilistic'
    confirmed_by_customer BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);
```

This design is system-agnostic: adding Salesforce or Xero support requires no schema migration — just new rows in `entity_links` with `source_system = 'salesforce'` or `source_system = 'xero'`. This aligns with the canonical data model approach described in Section 4.4 (Template Architecture), where connector modules map to a system-agnostic model rather than system-specific columns.

Every downstream layer (entity graph, semantic layer, AI agent) joins through these tables. The `match_confidence` and `confirmed_by_customer` fields allow downstream layers to handle uncertain matches appropriately — flagging metrics as approximate when computed from medium-confidence matches, for example.

Layer 4: Unified Entity Graph

What it does: Creates a single, cross-system view of every business entity by reading the `entities` and `entity_links` tables and joining data from all connected systems into rich, linked entity profiles that power both the semantic layer and the AI agent.

What's new here — and what isn't:

The unified entity graph is not a fundamentally new concept in data architecture. Enterprise data teams have built common data models (CDMs), master data management (MDM) systems, and Customer 360 platforms for decades. The underlying principles — entities, attributes, relationships, golden records — are well established.

What DataOrbit's unified entity graph is NOT:

- It is not a novel data modeling theory
- It is not a reinvention of MDM or CDM
- It is not a proprietary graph database technology

What IS new is three specific capabilities operating together in a way that SMBs have never had access to:

1. **Cross-system entity resolution without an MDM tool.** Enterprise MDM platforms (Informatica, Reltio, Tamr) cost hundreds of thousands of dollars and require data engineering teams to implement. DataOrbit performs entity resolution automatically when a customer connects their SaaS tools — no MDM tool, no data team, no implementation project.
2. **Derived entity states computed from multiple systems simultaneously.** No individual SaaS system can compute "customer health" because no individual system has the full picture. A customer's health depends on billing status (Stripe), support satisfaction (Zendesk), engagement recency (HubSpot), and payment history (QuickBooks). DataOrbit reads across all connected systems and derives these states in real time — something that previously required a data warehouse, custom SQL, and a BI tool.
3. **AI-readable entity context that powers natural language queries across all systems.** The entity graph isn't just a data store — it's the structured context that allows the AI agent to reason about business entities across system boundaries. When the CEO asks "How is Acme Corp doing?", the agent doesn't query four APIs — it reads one unified entity profile that spans CRM, billing, accounting, and support data. This is what makes cross-system Q&A possible without the AI hallucinating or guessing at joins.

The "graph" framing emphasizes the linked, multi-source nature of the data — which IS fundamentally different from a traditional CDM's flattened, single-schema approach. A CDM is a schema you design upfront and populate through ETL. DataOrbit's entity graph is a live resolution layer that links systems automatically, derives business states from the cross-system data, and powers an AI agent that reasons across all of them. The customer doesn't build it — it builds itself when they connect their tools.

The innovation is in the productization and accessibility, not in the underlying data modeling theory. DataOrbit makes entity-level, cross-system intelligence available to companies that have never had a data warehouse, never run an MDM project, and never employed a data engineer.

How to handle the inevitable question: If a prospect, investor, or technical evaluator asks "Isn't this just a common data model?" — the honest and compelling answer is: "*It draws from the same principles, yes. The difference is that a CDM is a schema you have to build and populate. This is a live entity resolution layer that links your systems automatically, derives business states from the cross-system data, and powers an AI agent that can reason across all of them. You don't build it — it builds itself when you connect your tools.*"

Entity types and cross-system enrichment:

Customer / Company Entity

The most important entity type. A unified customer profile combines:

Attribute	Source System	Example
Contact info, deal stage, assigned rep, activities	CRM (HubSpot)	Deal stage: "Negotiation," assigned to Sarah M.
Invoice history, payment status, lifetime revenue	Accounting (QuickBooks)	$47,200 lifetime revenue, $4,800 outstanding
Subscription status, MRR, plan tier, churn signals	Billing (Stripe)	Pro plan, $1,200/mo MRR, no failed payments
Open tickets, satisfaction score, last interaction	Support (Zendesk)	2 open tickets, CSAT 4.1/5, last contact 3 days ago

Derived Entity States

By reading across all systems, DataOrbit computes the entity's current state — something no single system can determine:

State	Cross-System Logic	Systems Required
Healthy	Active subscription + no open escalations + regular engagement + invoices current	Stripe + Zendesk + HubSpot + QuickBooks
At Risk	Active subscription + declining engagement OR open escalation OR overdue invoice	Stripe + (Zendesk OR HubSpot OR QuickBooks)
Expansion Ready	Healthy + usage approaching plan limits + no support issues + positive CSAT	Stripe + Zendesk + HubSpot
Churning	Cancellation requested OR subscription lapsed OR 30+ days no engagement	Stripe + HubSpot

States are derived from cross-system rules, not manually configured. The rules are pre-built per system combination template and can be customized through a guided UI.

Employee Entity (Growth tier)

Attribute	Source System
Name, department, role, manager, PTO balance	HRIS (BambooHR)
Assigned deals, pipeline value, win rate	CRM (HubSpot)
Assigned tickets, resolution time	Support (Zendesk)

Derived states: Overloaded, Underutilized, At Risk of Burnout, Ramping.

Deal Entity

Combines CRM deal data with billing and historical patterns. Derived states: On Track, Stalled, At Risk.

Design principle: Entity states are always current — as fresh as the last data sync. They don't require a separate configuration layer that can drift from reality. When data changes in any source system, the entity's state is recomputed at the next sync cycle.

Handling incomplete entity matching:

Not every entity will be matched across all systems. The entity graph must support partial profiles. A customer that's matched in HubSpot and Stripe but not yet in QuickBooks should still appear in the entity graph with the data available, and metrics should clearly indicate what's included: "Customer health score: 65 (based on CRM and billing data; accounting data not yet linked)."

Layer 5: Semantic Layer (Cube Core)

What it does: Defines a library of certified business metrics that span multiple source systems. Each metric has a precise definition, a calculation method, source system attribution, and a plain-language description. These metrics are the single source of truth for both the AI agent and any connected BI tools.

Technology choice: Cube Core (Apache 2.0)

We embed Cube Core as our semantic layer engine. We do not build a semantic layer from scratch.

Rationale:

- Apache 2.0 licensed — free to embed, modify, and distribute
- Connects to Postgres natively (our warehouse)
- Provides a Postgres-compatible SQL API (any tool that speaks Postgres can query it)
- Includes REST and GraphQL APIs for custom integrations
- Built-in caching via Cube Store for sub-second query response times

- Bundles DuckDB for lightweight analytical queries
- Supports YAML-based data model definitions (our metric templates)
- Production-proven — used by companies like Drata for AI-powered analytics
- AI API for LLM integration — purpose-built for agentic analytics

The semantic layer engine is commodity infrastructure. **What DataOrbit owns and differentiates on is the template library**: pre-configured Cube data models for specific system combinations.

Metric architecture:

Each metric is defined in Cube's YAML format with:

- **Name:** Human-readable ("Revenue Per Employee")
- **Definition:** Plain language explanation
- **Formula:** SQL computation referencing entity graph views
- **Source systems:** Which connected systems contribute data
- **Category:** Financial, Operational, People, Customer, Pipeline
- **Decomposition:** How it breaks down into sub-metrics

Template metric library: B2B SaaS (HubSpot + Stripe + QuickBooks)

This is the first template, containing approximately 30 certified metrics:

Financial Metrics:

Metric	Formula Concept	Sources
Monthly Recurring Revenue (MRR)	Sum of active subscription amounts	Stripe
Annual Recurring Revenue (ARR)	MRR × 12	Stripe
Net Revenue Retention (NRR)	Revenue from existing customers this period ÷ same customers prior period	Stripe + QuickBooks
Revenue Per Employee	Monthly Revenue ÷ Active Headcount	QuickBooks + BambooHR
Gross Margin	(Revenue – COGS) ÷ Revenue	QuickBooks
Pipeline-to-Revenue Ratio	Open Pipeline Value ÷ Monthly Revenue	HubSpot + QuickBooks
Average Collection Period	Mean days from invoice sent to payment received	QuickBooks

Customer Metrics:

Metric	Formula Concept	Sources
Customer Health Score	Composite: payment status (40%) + support satisfaction (30%) + engagement (30%)	QuickBooks + Zendesk + HubSpot
Customer Lifetime Value	Total revenue since first invoice	QuickBooks + HubSpot
At-Risk Revenue	Sum of MRR from customers in "At Risk" state	Derived (entity graph)
Time to First Value	Days from deal closed-won to first invoice paid	HubSpot + QuickBooks
Churn Rate	Customers lost ÷ customers at start of period	Stripe

Pipeline Metrics:

Metric	Formula Concept	Sources
Weighted Pipeline	Σ (Deal Amount × Stage Probability)	HubSpot
Pipeline Velocity	(# Deals × Avg Size × Win Rate) ÷ Avg Cycle	HubSpot
Stage Conversion Rates	Deals advancing ÷ total deals in stage	HubSpot
Coverage Ratio	Weighted Pipeline ÷ Remaining Quota	HubSpot + QuickBooks
Forecast Accuracy	Actual closed ÷ forecast at same point prior period	HubSpot + QuickBooks

People Metrics (when BambooHR connected):

Metric	Formula Concept	Sources
Quota Attainment	Rep closed revenue ÷ rep quota	HubSpot + QuickBooks + BambooHR
Capacity Score	Current workload ÷ standard workload × 100	HubSpot + BambooHR
Employee Turnover Impact	Revenue at risk from accounts assigned to departing employees	BambooHR + HubSpot + QuickBooks

Single-system vs. cross-system metrics:

This distinction is critical for the MVP. Single-system metrics (HubSpot pipeline, Stripe MRR, QuickBooks revenue) work without entity matching. Cross-system metrics (customer health, revenue per employee, NRR) require entity matching to be in place.

The product delivers value on day one with single-system certified metrics and progressively adds cross-system intelligence as entity matching validates and coverage increases. This means the Monday briefing works even if entity matching is still in progress — it just gets richer over time.

Influence maps (structured diagnostic framework):

Metrics are organized into decomposition trees that show how high-level outcomes break down into contributing drivers:

```
Revenue
├── New Revenue = New Deals × Avg New Deal Size
├── Expansion Revenue = Upsells + Cross-sells
├── Recurring Revenue = Prior MRR − Churn + Expansion
└── Churned Revenue = Lost Customers × Their MRR

Customer Health Score
├── Payment Status (40% weight)
│   ├── Days Since Last Payment
│   └── Outstanding Balance
├── Support Satisfaction (30% weight)
│   ├── CSAT Score
│   └── Open Escalations
└── Engagement Recency (30% weight)
    ├── Days Since Last Activity
    └── Feature Adoption %
```

These are mathematical decompositions, not causal claims. When revenue drops, the AI agent checks each branch: Did new deals decline? Did churn increase? This is arithmetic, not speculation.

Over time, as DataOrbit accumulates historical data, empirical patterns emerge (e.g., "Customers with 2+ unresolved escalations for >7 days have churned within 90 days in 68% of observed cases"). These are transparently labeled with confidence level, observation count, and last validation date. The AI never presents a correlation as a confirmed cause.

Layer 6: AI Agent — DataOrbit Apollo

What it does: Reads the semantic layer (via Cube's SQL/REST API), the entity graph, and the influence maps. Produces three types of output:

Output 1: The Weekly Business Briefing

Delivered Monday morning (or custom schedule), via email, Slack, or in-app. Structured as a concise executive summary covering:

- **Headline:** One-sentence state of the business
- **Financial Snapshot:** Revenue vs. target, margin, cash position
- **Pipeline:** Coverage ratio, stage analysis, gap identification

- **Customers:** Health distribution, at-risk customers with context, expansions
- **Team:** Headcount, capacity, overloaded individuals, PTO impacts
- **Watch List:** What needs attention this week

Example:

```
DATAORBIT APOLLO — WEEKLY BRIEFING
Week of February 10, 2026

——— HEADLINE ———
Revenue is on track. Pipeline needs attention. Two customers showing
early warning signs.

——— FINANCIAL SNAPSHOT ———
Monthly Revenue: $487K (94% of $520K target)
  → Recurring: $412K (stable)
  → New: $75K (below $108K target — see Pipeline)
Gross Margin: 68.2% (up from 66.8% last month)

——— PIPELINE ———
⚠️ Pipeline coverage is 1.8× (target: 3×)
Weighted pipeline: $194K against $108K remaining target
  → 4 deals in Proposal ($142K) — need 76% conversion
  → Historical conversion at this stage: 34%
  → At current rates, expect ~$48K new revenue vs. $108K target

——— CUSTOMERS ———
✅ 142 of 147 active customers Healthy
⚠️ 3 customers moved to At-Risk:
  1. TechFlow Inc ($4,200/mo) — 2 support escalations unresolved >7 days
     CSAT: 4.4 → 2.8. Account manager: David K.
  2. Meridian Labs ($2,800/mo) — no product login in 18 days
     Last invoice paid 5 days late. Account manager: Sarah M.

——— TEAM ———
Sarah M. carrying 16 active deals (team avg: 11). She is also
account manager for Meridian Labs (at-risk). Capacity constraint
may be contributing to both pipeline delays and at-risk account.

——— WATCH LIST ———
- Pipeline gap widens if Proposal deals don't advance by Feb 21
- TechFlow escalations need resolution this week to prevent churn
```

The key characteristic: cross-system insights that no single dashboard could provide. "Sarah is overloaded AND her at-risk customer is underserved" requires CRM + HRIS + Support data simultaneously.

Output 2: Exception Alerts

Delivered same day (within hours of detection), via email, Slack, or push notification. Triggered when a certified metric crosses a configurable threshold or when cross-system signals converge.

Alert anatomy includes: what happened, cross-system context, historical pattern (if applicable with confidence level), and suggested attention areas.

Pre-built alert thresholds with sensible defaults:

Alert Type	Default Threshold
Customer enters At-Risk	Automatic (derived from cross-system rules)
Revenue below plan pace	<85% of target at midpoint
Pipeline coverage drops	Below 2.5× remaining quota
Support escalation unresolved	>5 business days
Employee overload detected	>140% of team average workload
Payment failure	Any failed payment
Churn event	Any cancellation or non-renewal

Rule-based alerts work immediately on day one. Statistical alerts (unusual metric movements) require 2-4 weeks of baseline data and are suppressed until then.

Output 3: On-Demand Q&A

Interface: chat (in-app, Slack, or email). The user asks a question in plain language. The agent maps it to certified metrics, queries the semantic layer, and synthesizes a response citing which systems contributed.

Examples by role:

CEO: "How are we tracking against Q1 targets?" → Agent synthesizes revenue vs. plan (QuickBooks), pipeline coverage (HubSpot), headcount vs. hiring plan (BambooHR), customer health distribution. One answer from four systems.

VP of Sales: "Why is Mike's pipeline lower than the rest of the team?" → Agent responds with context: Mike closed 3 deals last month (above average), so his pipeline is low because he converted more. His new pipeline generation is on track. He was also on PTO for 4 days in January (BambooHR).

CFO: "Which customers are at risk of late payment this quarter?" → Agent identifies 7 overdue invoices + 3 customers with a historical pattern of late payment. Combined exposure: $84,200.

What the agent explicitly does NOT do:

- Make autonomous decisions or take actions in source systems

- Present correlations as confirmed causal relationships
- Speculate beyond what the data shows — if it doesn't know, it says so
- Generate insights from data it doesn't have current access to

3.3 Design Rationale: Why Each Layer Is Designed This Way

The six-layer architecture described above emerged from a deliberate process of critique, correction, and honest reassessment. Several foundational design choices were challenged, revised, or reframed during this process. This section documents the reasoning behind each layer's design — not just what it does, but why it's designed this way and what alternatives were rejected.

This matters because anyone building, extending, or evaluating this product will inevitably ask the same questions we already asked ourselves. The answers are here.

The Data Quality Engine: Why a Dedicated Layer, Not a Feature

Early versions of the architecture treated data quality as a concern handled within the connector layer — basic validation checks during ingestion. This was inadequate for three reasons.

First, data quality is not just an ingestion problem. Quality degrades at every boundary in the pipeline: entity matching can produce false positives, metric computation can amplify small errors, and the AI agent can narrate stale data with false confidence. A quality check at ingestion catches source problems but misses downstream propagation. The Data Quality Engine monitors every boundary — not just the first one.

Second, for the target market, data quality is the primary trust barrier. SMB leaders who have been burned by wrong numbers in spreadsheets will not trust an AI analyst unless it can demonstrate that it knows when its own data is reliable and when it isn't. Confidence tagging on every output — "Your MRR is $412,000 (high confidence)" vs. "Revenue Per Employee is $3,420 (low confidence — stale HRIS data)" — is the mechanism that builds trust. This is a product feature, not an engineering concern. It deserves a dedicated layer because it's customer-facing and visible.

Third, downstream impact analysis — tracing a data quality issue through entity matching → metrics → AI output → quantified business impact — is something no standalone data observability tool can do. Tools like Monte Carlo, Soda, or Great Expectations monitor data quality in isolation. They can tell you "this column is 40% null." They cannot tell you "this 40% null rate means $47K of revenue is not being monitored for churn risk." DataOrbit can, because it owns the entire pipeline. This full-pipeline impact trace is the genuinely novel capability, and it only works as a dedicated layer with visibility into all downstream layers.

The alternative considered was embedding quality checks into each layer independently — the connector layer checks source quality, the entity matching layer checks match quality, the semantic layer checks metric quality. This was rejected because it fragments the quality story. The customer would see quality information scattered across different screens with no unified view and no way to trace an issue from root cause to business impact. A dedicated layer with a single Pipeline Health view, a System Health view, and an Impact view provides the unified quality narrative that builds customer confidence.

The Unified Entity Graph: Why "Graph," Not "State Machine"

Early versions of the architecture described the entity layer as a "state machine" — a system that tracks each

entity through defined lifecycle states (Lead → Customer → At-Risk → Churned). This framing was rejected for two reasons.

First, it invited a well-founded technical critique: operational systems already manage their own states. HubSpot tracks deal stages. Stripe tracks subscription status. Zendesk tracks ticket lifecycle. Building another state machine on top of these systems would mean replicating OLTP state management — a solved problem — rather than solving the unsolved problem of cross-system synthesis.

Second, "state machine" sounds like computer science, not business value. It communicates implementation detail rather than customer outcome.

The unified entity graph framing solves both problems. The entity graph doesn't replicate any individual system's state management. Instead, it reads the current state from every connected system and synthesizes a cross-system picture that no single system can produce on its own.

A customer entity in the unified graph has: their CRM record (contact info, deal stage, last activity, assigned rep), their accounting record (invoices, payment status, revenue, outstanding balance), their support record (open tickets, satisfaction score, last interaction), and their HRIS-adjacent data (which employee is their account manager, is that employee on leave or overloaded). No single source system has this complete picture.

Lifecycle states emerge naturally from this graph. A customer whose deal stage is "Closed Won" in CRM, whose first invoice is "Paid" in accounting, and whose onboarding ticket is "In Progress" in support is clearly in an "Onboarding" state. DataOrbit derives this from cross-system data rather than requiring hand-configured YAML state definitions that can drift from reality. The state definitions become rules over the unified entity graph, not independent configuration.

This is the genuine, defensible innovation: no single system knows the full state of the entity. DataOrbit does, because it is the only layer that sees across all systems simultaneously.

The Semantic Layer: Why Dramatically Simpler Than Enterprise Approaches

For this market, the semantic layer should be dramatically simpler than what enterprise platforms offer. These customers don't need hundreds of metrics. They need 20-40 well-defined cross-system metrics that they currently can't get without manual spreadsheet work.

The key insight for this market: the metrics themselves are largely standardized. "Revenue per employee" means the same thing at every 200-person company running QuickBooks and BambooHR. "Pipeline-to-close conversion rate" is calculated the same way for every company using HubSpot. This means template-based metric libraries per system combination can cover 80% of what the customer needs out of the box.

These metrics fall into natural categories that cross system boundaries:

- **Financial metrics** cross accounting + CRM: revenue per customer, pipeline-to-revenue conversion, average deal cycle time, revenue per employee
- **Operational metrics** cross CRM + support + project management: time to onboard, support tickets per customer per month, project utilization rate
- **People metrics** cross HRIS + CRM + project management: revenue per rep, quota attainment, rep capacity vs. assigned pipeline, employee turnover impact on account coverage

The remaining 20% — custom fields, non-standard workflows, industry-specific metrics — is where light configuration or guided setup fills the gap. But the starting point is pre-built, not blank. This is a fundamental product decision: DataOrbit ships with answers, not with a blank canvas and a configuration manual.

Influence Maps: Why Not "Causal Graphs"

Early versions of the architecture described the diagnostic layer as a "causal graph" — a system that identifies which variables cause changes in business outcomes. This framing was rejected because it overclaims.

"Causal graph" implies scientific rigor and validated causation — the kind of claim that requires controlled experiments, counterfactual reasoning, and statistical validation that a business intelligence platform cannot deliver. It invites legitimate scrutiny from anyone with a statistics background, and it sets expectations the product cannot meet.

The reframing to "influence maps" is deliberate and precise. An influence map says: "These metrics are related and worth investigating when something changes." It sets the right expectation: the system directs your attention to the most likely contributing factors, not delivers a proven causal chain.

For SMBs with standardized systems, the influence maps are well-understood and relatively stable:

- When revenue drops, the likely factors are: pipeline volume, deal conversion rate, average deal size, and sales cycle length. These aren't controversial causal claims — they're the standard decomposition of a revenue equation.
- When customer satisfaction drops, the likely factors are: support ticket volume, first-response time, resolution time, and product usage trends.

The influence map is organized as a metric decomposition tree rather than an arbitrary graph. Revenue decomposes into pipeline × conversion × deal size. Each of those decomposes further. The structure is algebraic — it's the mathematical relationship between the metrics, not a statistical assertion about causation. When an output metric changes, the system checks each driver in the decomposition tree to see which one moved. That's not causal inference — it's structured diagnostic reasoning. And it's rigorous because the decomposition is mathematically grounded.

Where it gets more interesting: some influence relationships are empirical rather than algebraic, such as "employee turnover on the CS team correlates with increased churn in the following quarter." For these, DataOrbit is transparent: it tags them as "observed pattern" versus "mathematical relationship" and shows the historical evidence. Over time, as the system accumulates data, these empirical patterns either strengthen (increasing confidence) or weaken (triggering a review). This gives the system a form of pattern confidence monitoring — but framed honestly as pattern strength rather than causal validation.

The Pattern Intelligence Flywheel: Near-Term Feature, Long-Term Moat

The pattern confidence monitoring described above is not just a feature of the influence map layer. It is the data collection infrastructure for DataOrbit's long-term defensibility — and, ultimately, for the thesis that every business will have its own intelligence model trained on its own operational data.

This section documents that thesis honestly: what's real now, what requires scale, and what the long-horizon strategy looks like.

The core thesis: Every time DataOrbit observes a pattern — "customer with 2+ unresolved escalations for >7 days churned within 90 days" — and tracks whether that pattern held or didn't, it is building a labeled dataset. Every confirmed alert ("this was useful"), every dismissed alert ("this was noise"), every customer outcome that validates or contradicts a predicted state — all of this is training data. Not in a vague, aspirational sense. In the concrete sense that it produces structured observations linking cross-system signals to business outcomes, with timestamps, confidence scores, and feedback labels.

That dataset is the raw material for any future model — whether it's a statistical model, an ML classifier, or structured context for an LLM. The compounding accuracy thesis is directionally correct: a pattern observed 8 times has weak confidence; the same pattern observed 200 times with a 68% hit rate has strong confidence; at 2,000 observations, you can start segmenting by industry, company size, or system combination.

The honest constraint: individual SMB data volumes are tiny.

A 200-person B2B SaaS company might have 300 customers, 15 sales reps, and 50 deals closing per quarter. That is not enough data to train a meaningful per-business model. "Employee turnover on the CS team correlates with increased churn" — how many times does a CS rep leave at a 200-person company? Maybe twice a year. You'd need a decade of data to have 20 observations, and by then the business has changed so fundamentally that the early data points may be irrelevant.

This small-data problem is real. It doesn't invalidate the thesis — it redirects it.

Where the real flywheel lives: aggregate patterns across customers.

The compounding intelligence doesn't come primarily from each customer's individual data. It comes from aggregate patterns across all customers on the same system combination. This is the critical strategic distinction:

Per-customer learning is weak in the near term but valuable long term. "At Acme Corp specifically, when support tickets exceed 3/month for a customer, that customer churns within 90 days." This requires Acme Corp to have enough customers and enough history for the pattern to be statistically meaningful. For most SMBs, this takes 12-24 months of data accumulation before it produces useful signal.

Cross-customer aggregate learning is strong much sooner, and this is the moat. "Across 100 B2B SaaS companies running HubSpot + Stripe + Zendesk, when support tickets exceed 3/month for a customer, that customer churns within 90 days in 68% of cases." This is powerful because DataOrbit sees the same pattern play out across hundreds of businesses with similar structures. Each individual customer contributes a few data points. The aggregate contributes thousands.

The five-phase flywheel:

Phase 1 — Launch (Day 1): Pattern templates are pre-built from established business logic. "Revenue = Pipeline × Conversion × Deal Size" is algebraic — no data needed. Empirical patterns are seeded from well-established industry benchmarks and clearly labeled as "industry baseline, not yet validated with your data."

Phase 2 — Early Customers (Months 3-12): Per-customer empirical patterns begin accumulating. Confidence is low because sample sizes are small. Patterns are transparently tagged: "Observed 8 times at your company, held in 5 cases (62% accuracy)." The product value at this stage is transparency and attention-direction, not prediction.

Phase 3 — Aggregate Intelligence (Months 12-24, 50+ customers): Aggregate patterns across customers on the same system combination reach meaningful sample sizes. DataOrbit can now tell a new customer on day one: "Across companies like yours using HubSpot + Stripe + Zendesk, this pattern holds with 74% confidence based on 400+ observations." New customers get pre-loaded empirical patterns from their first briefing — not just algebraic decompositions. This is a genuine network effect: each new customer makes the patterns more accurate for every other customer on the same stack.

Phase 4 — Predictive Models (Months 24-36, 200+ customers): Enough aggregate data exists to train actual predictive models per vertical and per system combination. "B2B SaaS companies with this engagement profile have a 3× higher churn probability in the next quarter." Individual customer models become feasible as their own data accumulates alongside aggregate priors — the aggregate patterns serve as Bayesian priors that the customer's own data refines.

Phase 5 — Per-Business Intelligence Models (Long term, 500+ customers): Each business has its own intelligence model, initialized with aggregate priors from similar companies and continuously refined with their own operational data. The model improves with every data sync, every confirmed alert, every piece of feedback. This is the "every business has its own trained model" vision — and it's credible at this scale because the aggregate data solves the cold-start problem and each customer's accumulating history adds the specificity.

What this means for engineering decisions today:

The full value of this flywheel won't materialize until Phase 3 or 4. But the data collection infrastructure must be built from day one. Specifically:

- Every empirical pattern observation must be stored with: the pattern definition, the observation timestamp, whether the predicted outcome occurred, the confidence at time of observation, and the customer's system combination and vertical.
- Every alert must capture feedback: "useful," "noise," or "not acted upon" (implicit signal).
- Every entity state transition must be logged: when a customer moved from Healthy to At-Risk, what the cross-system signals were at that moment, and what happened next (churned, recovered, escalated).
- All of this data must be structured for future aggregation across customers — anonymized, normalized by system combination and vertical, and queryable.

If DataOrbit doesn't collect this data from its first customer, it can't build aggregate pattern intelligence when it reaches 50 or 100 customers. The data collection is cheap. The opportunity cost of not collecting it is enormous.

How to talk about this externally:

To customers: "DataOrbit gets smarter about your business over time. The longer you use it, the more accurately it identifies what matters and what's noise."

To investors: "Every customer on the same system combination contributes to a shared pattern intelligence layer. This creates a data network effect — each new customer makes the product more accurate for every existing customer. After 100 customers on HubSpot + Stripe + Zendesk, our pattern library for that stack is something no new entrant can replicate without years of data accumulation."

To engineers: "Build the observation logging infrastructure in Phase 1. It's a few database tables and some event tracking. The models come later. The data must start accumulating now."

The AI Agent: Why Synthesis Over Autonomy

For SMBs, a proactive AI agent is actually more valuable than it is for enterprises, precisely because SMBs don't have data teams to watch dashboards. The CEO of a 200-person company doesn't have an analytics team producing weekly reports. They check dashboards sporadically, react to problems late, and rely on gut feel.

But the agent's design is deliberately constrained. Early versions of the architecture described an agent that autonomously traverses a causal graph and generates confident recommendations. This was rejected. Instead, the agent synthesizes cross-system data into a clear narrative and highlights the most likely drivers. The human makes the decision. The agent makes sure the human has the complete picture — from all systems, in one place, in plain language.

The agent's three outputs are prioritized in a specific order that reflects this market's needs:

Priority 1: The Monday Morning Briefing. This is the thing the CEO currently builds manually in a spreadsheet on Sunday night — or more likely, doesn't build at all. A weekly synthesized report that says: "Here's where your business stands across all systems. Revenue is on track. Pipeline has a gap in the enterprise segment. Two customers are showing early warning signs. Your CS team is at 90% capacity — if both escalate, you'll need to deprioritize something." This single output replaces 3-8 hours of manual weekly work.

Priority 2: Exception Alerts. Not continuous 24/7 monitoring — that's overkill for this market. Rather, daily checks against configurable thresholds, enriched with cross-system context. "Customer satisfaction for Acme Corp dropped from 4.5 to 3.2 — two open support tickets unresolved for 9 days." The cross-system context is the value — no single system can fire these alerts because no single system has the complete picture.

Priority 3: Question Answering. When the CEO gets the Monday briefing and wants to dig deeper: "Why is Sarah's team underperforming?" The agent queries the unified entity graph, checks the influence map, and responds: "Sarah has 3 reps. One has been on leave since January 15 (HRIS data). The other two are carrying 140% of normal pipeline load (CRM data). Their conversion rate hasn't dropped — they just have fewer at-bats. The issue is capacity, not performance." This is the cross-system insight that no single BI tool can provide.

This design is more honest, more defensible, and more useful for this market. SMB leaders don't want an AI making decisions for them. They want an AI that saves them 5 hours of spreadsheet work and makes sure they don't miss something important.

The BI Integration: Why One Source of Truth Matters

The cross-system metrics need to be consumable in the customer's BI tool of choice — or in a native DataOrbit dashboard if they don't have one. The value here is simple and inarguable: certified cross-system metrics available as a data source in an existing BI tool. The customer can build whatever dashboards they want, with the confidence that "revenue per employee" in their Power BI report is pulling from the same governed definition that the AI agent uses in the Monday briefing.

This is where the semantic layer value is clearest and least controversial. No one disputes that having one definition of "revenue per employee" serving both the AI and the dashboard is better than having them calculated differently in two places.

3.4 BI Integration Layer

DataOrbit exposes its certified metrics as a data source that any standard BI tool can connect to. Cube Core's Postgres-compatible SQL API means that Metabase, Power BI, Tableau, Looker Studio, or any SQL-capable tool can query DataOrbit's governed metric layer directly.

What flows to the BI tool: all certified metrics, entity-level attributes, derived states.

What does NOT flow: AI narratives (those live in Apollo), influence map reasoning (those power the agent, not dashboards), raw source system data (the BI tool sees governed metrics, not raw tables).

The key value: the same metric definition that powers the AI agent's Monday briefing also powers the CFO's Power BI report. One source of truth.

3.5 AI Capabilities Across Every Layer

The AI agent (Layer 6) is the most visible AI capability in the product — it generates briefings, fires alerts, and answers questions. But treating AI as a feature of only one layer is a strategic mistake. Every layer in the architecture has specific opportunities for AI to either augment the engineering work behind the scenes (reducing our build cost and improving reliability) or deliver customer-facing intelligence that would be impossible without it.

This section maps AI capabilities to each layer, distinguishing between two types:

- **Behind-the-scenes AI** — augments our engineering, reduces manual configuration, improves system accuracy. The customer never sees these directly but benefits from faster setup, better data quality, and more accurate matching.
- **Customer-facing AI** — delivers capabilities or intelligence that the customer interacts with. These are product features, not infrastructure improvements.

The principle: AI should be woven into the architecture from the ground up, not bolted on at the top.

Layer 1: Managed Warehouse + Data Connectors

Behind-the-scenes AI:

Intelligent schema mapping. When a new connector is added or a SaaS vendor changes their API schema, AI can infer how new or changed fields map to DataOrbit's internal data model. Today, this is hard-coded per connector — if HubSpot adds a new property type or QuickBooks restructures their invoice endpoint, an engineer has to update the mapping manually. An LLM that has seen thousands of API schemas can suggest mappings automatically: "This new field `hubspot.companies.annual_revenue_v2` appears to replace the deprecated `hubspot.companies.annualrevenue` — recommend mapping to the same internal field with a type conversion." This reduces connector maintenance cost significantly as the number of supported systems grows.

Data quality anomaly detection. After each sync, AI can scan the ingested data for anomalies that indicate something has gone wrong upstream — not in DataOrbit, but in the customer's source system or in the ingestion pipeline itself. Examples: record count drops 40% compared to the previous sync (suggesting an API pagination failure or a bulk delete in the source system), a field that was previously populated in 95% of records is now null in 60% (suggesting a source system configuration change), or a date field suddenly contains values from

1970 (suggesting a timezone or epoch conversion error). These checks prevent garbage data from flowing into the entity graph and corrupting downstream metrics. Without AI, you'd need hand-written rules for every possible anomaly pattern. With AI, you can detect novel anomalies that no one anticipated.

Smart sync scheduling. Not all data changes at the same rate. Deal stages in HubSpot change daily. Company metadata changes rarely. Employee records in BambooHR change infrequently except during hiring surges. AI can learn the change frequency of each data type per customer and optimize sync scheduling — syncing high-change-rate data hourly and low-change-rate data daily or weekly. This reduces API call volume (important for rate-limited APIs), reduces compute cost, and ensures that the data that matters most is always the freshest.

Customer-facing AI:

Conversational connector setup. Instead of a purely form-based OAuth flow, AI can guide the customer through connection setup conversationally: "I can see you're connecting QuickBooks. Do you use classes to track departments? Do you want me to pull expense data in addition to revenue?" This replaces a static configuration wizard with an adaptive one that asks relevant questions based on what it discovers about the customer's system configuration during the OAuth handshake and initial data scan.

Data health dashboard narration. After the initial data load, AI can generate a plain-language summary of what it found: "We pulled 347 companies from HubSpot, 289 customers from QuickBooks, and 312 customers from Stripe. 78% of HubSpot companies have a domain name filled in (this helps with matching). 22% are missing domain — we'll rely on email and company name for those. Your QuickBooks data looks clean — no obvious duplicates detected." This gives the customer immediate confidence that the system understood their data, and surfaces any issues transparently rather than burying them in a log file.

Layer 2: Data Quality Engine

Behind-the-scenes AI:

AI-powered anomaly detection. Instead of hard-coded rules ("alert if record count drops more than 20%"), AI learns the normal patterns of each data source and each customer. HubSpot data for a 200-person B2B SaaS company typically has 200-500 company records, updated 5-15 per day. If today's sync shows 50 updates, that's unusual but not alarming (possibly end of quarter activity). If it shows 0 updates, that's likely a sync failure. If it shows 5,000 updates, that's likely a bulk import or data migration. The AI characterizes the anomaly rather than just flagging it — explaining what's unusual and what the most likely cause is.

AI-powered root cause analysis. When a metric changes unexpectedly, AI traces backwards through the pipeline: "Revenue Per Employee dropped 34%. Checking revenue... unchanged. Checking headcount... BambooHR shows 4 new employees added this week. Root cause: headcount increase diluted the ratio, not a revenue problem." This multi-step diagnostic follows the influence map backwards, checking each driver — exactly the kind of structured reasoning the Query Pipeline was designed for.

AI-powered remediation suggestion. "32 HubSpot companies are missing domain. We can auto-populate 18 of these by extracting domains from contact email addresses already in HubSpot (e.g., jane@acme.com → acme.com). Would you like us to suggest these updates?" The AI doesn't modify source system data — that's the customer's responsibility. But it identifies the fix and makes it actionable.

Customer-facing AI:

Downstream impact narration. The Data Quality Engine's most valuable AI capability: generating plain-language explanations of how a data quality issue affects every downstream layer. "83 companies missing domain → entity match coverage drops from 89% to 78% → $47K in revenue excluded from cross-system metrics → At-Risk detection blind spot for those customers." No standalone data quality tool can produce this because it requires knowledge of the entity matching rules, metric definitions, and AI agent behavior — all of which DataOrbit owns.

Confidence tagging on AI outputs. Every number the AI agent reports carries a quality indicator computed from upstream quality signals. The Data Quality Engine provides the confidence scores; the AI agent narrates them naturally: "Your MRR is $412,000 (high confidence — Stripe synced 2 hours ago)" vs. "Revenue Per Employee is $3,420 (low confidence — BambooHR sync is 3 days stale)." This builds trust by being transparent about what the system knows well versus what it's uncertain about.

Data quality improvement coaching. AI identifies the highest-impact data quality improvements and explains them in business terms: "The single biggest improvement you can make to DataOrbit's accuracy is populating the domain field for 83 HubSpot companies. This would increase cross-system monitoring from 78% to an estimated 89% of your customer base and bring an additional $47K/month under churn risk detection." This turns data quality from a technical concern into a business priority with quantified impact.

Layer 3: Entity Matching Engine

Behind-the-scenes AI:

Learning from human confirmations. Every time a customer confirms a match ("Yes, Acme Corp = ACME LLC") or rejects one ("No, these are different companies"), that feedback is a training signal. AI can learn which features of a match candidate predict human confirmation versus rejection. Over time, this shifts ambiguous matches from "needs human review" to "auto-matched with high confidence" — reducing the customer's onboarding burden for their second, third, and hundredth data sync. Across customers, these learnings compound: if 50 customers on HubSpot + QuickBooks have all confirmed that stripping "LLC" from company names produces valid matches, the system can do this automatically for the 51st customer.

Candidate pair generation. Rule-based matching (deterministic and probabilistic) can only find matches based on fields it's been told to compare. AI can discover candidate pairs that rule-based systems miss. For example: a customer's QuickBooks has "John Smith Consulting" and HubSpot has a company called "JSC Partners" with a contact named John Smith. A rule-based system comparing company names would never match these. An LLM evaluating the full context — company name, contact names, email domains, industry, location — can flag this as a possible match worth human review. This is the Tier 4 LLM-assisted matching described in the architecture, but the key insight is that it's not a future nice-to-have — it's the mechanism by which entity match coverage improves from 70% to 85%+ over time.

Match staleness detection. Entities change. Companies merge, rebrand, or split. An employee leaves one customer and joins another. A match that was correct six months ago may not be correct today. AI can monitor the entity graph for signals that a previously confirmed match may have gone stale — for example, if two matched entities suddenly have completely divergent activity patterns, different addresses, or different contact names in all fields. Flagging these for re-review maintains match precision over time, which is critical because a single false positive (wrong merge) contaminates every downstream metric.

Customer-facing AI:

Natural language match review. Instead of presenting match candidates as a raw data table ("HubSpot Company: Acme Corp, Domain: acme.com | QuickBooks Customer: ACME LLC, No domain"), AI can explain the match in plain language: "We're fairly confident these are the same company. The name is similar after removing 'LLC,' and both have invoices from the same billing contact (jane@acme.com). However, the addresses don't match — HubSpot shows San Francisco and QuickBooks shows New York. This could be a branch office situation. Can you confirm?" This dramatically reduces the cognitive load of match review, especially for customers with dozens of ambiguous matches during onboarding.

Match coverage reporting. AI generates a natural language summary of entity matching status: "85% of your HubSpot companies are linked to QuickBooks customers. 12% are under review — we've found possible matches but need your confirmation. 3% have no match candidates in QuickBooks, which likely means they're prospects who haven't become paying customers yet." This is more useful than a raw percentage because it explains what the numbers mean in business terms.

Layer 4: Unified Entity Graph

Behind-the-scenes AI:

Conflict resolution. When two source systems disagree about the same attribute — HubSpot says the company is in San Francisco, QuickBooks says New York — a rule-based system applies a static hierarchy (e.g., always prefer QuickBooks for addresses). AI can make smarter decisions by considering context: if the QuickBooks address was last updated 2 years ago and the HubSpot address was updated last week, the HubSpot address is probably more current. If the customer has contacts in both cities, it might be a multi-location company. AI can flag genuinely ambiguous conflicts for human review rather than silently applying a rule that may be wrong.

Derived state rule suggestion. The initial derived states (Healthy, At Risk, Expansion Ready, Churning) are pre-configured per system combination template. But as DataOrbit accumulates data about actual customer outcomes — which customers actually churned, which actually expanded — AI can suggest new state rules or refinements to existing ones. "Customers who have been in 'Healthy' state but whose product login frequency dropped by 50% over 30 days churned within 60 days in 73% of observed cases. Consider adding a 'Silent Risk' state for this pattern." This connects directly to the pattern intelligence flywheel: the AI uses accumulated observations to suggest rule improvements that make the entity graph more predictive over time.

Customer-facing AI:

Natural language entity summaries. This is one of the highest-value AI capabilities in the product. For any entity in the graph, AI generates a plain-language summary that synthesizes all cross-system data into a narrative:

"**Acme Corp** has been a customer since March 2024. They're on the Pro plan at $4,200/month (Stripe), up from $2,800 when they started — they expanded in September. Their account manager is David K., who currently manages 9 accounts. Payment history is clean — every invoice paid within 15 days (QuickBooks). However, they've opened 3 support tickets this month, up from an average of 0.5 per month (Zendesk). Two are unresolved. Their primary contact hasn't logged into the product in 11 days (HubSpot activity). Current state: **At Risk** — the combination of rising support volume and declining engagement is a pattern we've seen precede churn in similar companies."

No dashboard can produce this. No single system has the data to write it. This is the unified entity graph made conversational — and it's the kind of output that makes a CEO say "this is worth paying for."

"What changed?" entity diffs. When the AI agent flags an entity that has changed state — a customer moving from Healthy to At Risk — AI can generate a precise explanation of what changed across all systems since the last review: "Since last Monday: 2 new support tickets opened (both unresolved), product login frequency dropped from daily to zero, and the primary contact hasn't responded to the last 3 emails from their account manager. Payment is still current. The trigger for the state change was the combination of support escalation and engagement drop."

Layer 5: Semantic Layer (Cube Core)

Behind-the-scenes AI:

Metric definition suggestion. When a customer connects a new system or DataOrbit encounters data patterns it recognizes, AI can suggest additional metric definitions: "You've connected BambooHR in addition to HubSpot and QuickBooks. This enables 8 additional cross-system metrics including Revenue Per Employee, Quota Attainment, and Employee Turnover Impact on Account Coverage. Would you like to activate these?" Rather than presenting a static catalog of available metrics, the system proactively identifies what's now possible given the customer's connected systems.

Auto-generated metric documentation. Every metric definition in Cube is a YAML file with a formula. AI can generate plain-language documentation for each metric automatically: "**Revenue Per Employee** is calculated by dividing total monthly revenue (from QuickBooks paid invoices) by active headcount (from BambooHR employees with an active status). This metric updates daily. A rising trend means the company is generating more revenue per person — either through growth or efficiency. A declining trend could indicate over-hiring relative to revenue growth, or revenue decline without corresponding headcount reduction." This documentation powers the AI agent's explanations and gives customers transparency into exactly how each metric is computed.

Anomaly detection on metric values. Before a metric reaches the AI agent for narration, the semantic layer can use AI to flag statistical anomalies: "Revenue Per Employee dropped 23% this week, which is 2.4 standard deviations below the 90-day moving average. This is the largest single-week movement in the last 6 months." This pre-computation means the AI agent doesn't have to decide what's noteworthy — the semantic layer surfaces it, and the agent narrates it.

Customer-facing AI:

Natural language metric exploration. Powered by Cube's AI API, customers can ask questions about metrics in plain language: "How is revenue per employee trending compared to last quarter?" The AI translates this to a Cube query, retrieves the data, and responds conversationally. This is distinct from the Layer 5 Q&A capability — that answers business questions by synthesizing across metrics. This answers questions about the metrics themselves: what they measure, how they're computed, why they changed, and how they compare to historical values.

Metric relationship discovery. As the metric library grows per customer, AI can identify relationships between metrics that aren't in the pre-built influence maps: "Over the last 3 months, your Support Ticket Volume and Churn Rate have moved together with a 2-week lag — when tickets spike, churn follows about 14 days later. This pattern has appeared 4 times. Confidence: moderate." This is the influence map growing organically through AI observation rather than through static configuration.

Layer 6: AI Agent (DataOrbit Apollo)

Layer 6 is already AI-native by design — it generates briefings, alerts, and Q&A responses using an LLM. But there is a meaningful distinction between the current architecture (the LLM narrates pre-computed metrics) and a deeper integration where AI actively reasons about the intelligence it delivers.

Behind-the-scenes AI:

Adaptive briefing content. Rather than generating the same briefing structure every week, AI can learn which sections the customer actually reads, clicks on, and acts upon. If the CEO consistently engages with the Pipeline section and ignores the Team section, the briefing can lead with pipeline intelligence and compress the team section. If the CEO frequently asks follow-up questions about customer health after reading the briefing, the next briefing can proactively include more detail in that section. This isn't just personalization — it's the briefing becoming more useful over time as it learns what the customer cares about.

Alert calibration through feedback. Every alert the customer marks as "useful" or "noise" is a training signal. Over time, AI can adjust alert thresholds and relevance scoring per customer: "This customer dismisses alerts about small payment delays but always acts on support escalation alerts — weight support signals higher in their alert ranking." This directly reduces alert fatigue, which is the primary risk to the alert system's long-term value.

Multi-step reasoning. When the AI agent answers a complex question ("Why are we behind on Q1 targets?"), it currently does a single-pass query of the semantic layer and narrates the results. A more capable version would perform multi-step reasoning: first query revenue vs. target, then identify which component (new, expansion, recurring, churned) is underperforming, then drill into that component's drivers using the influence map, then check for cross-system context (is the sales team short-staffed? are key accounts at risk?). Each step informs what to query next. This transforms the agent from a narrator into a genuine analyst that follows threads of investigation.

Customer-facing AI:

Proactive insight generation. Beyond structured briefings and threshold-based alerts, AI can generate unstructured insights — observations that don't fit neatly into a metric threshold but are worth noting: "Three of your top 10 customers by revenue have had their account managers change in the last 60 days. Historically, account manager transitions are associated with a temporary dip in customer satisfaction scores. You may want to schedule check-in calls with these accounts." This is the AI acting as a thoughtful analyst who notices patterns a dashboard wouldn't surface.

Conversational deep-dives. After receiving a briefing or alert, the customer can ask follow-up questions in a continuous conversation thread. The AI maintains context: "You mentioned Sarah's team is behind on pipeline. Is that a new trend or has it been declining?" → "Sarah's pipeline has been declining for 3 consecutive weeks. Prior to that, it was above team average. The decline started the same week she took over the Meridian Labs account, which has required significant attention due to 4 open escalations." This thread-based reasoning across multiple turns transforms Q&A from a single-query lookup into an actual analytical conversation.

Scenario simulation. The customer asks "What happens to our revenue if we lose Acme Corp?" The AI computes: Acme Corp's MRR ($4,200), their percentage of total revenue (3.2%), the impact on NRR if they churn, the pipeline coverage gap this would create, and which customers have similar risk profiles. This isn't prediction — it's structured what-if analysis using certified metrics. "If Acme Corp churns, monthly revenue

drops to $483K. Your NRR would fall from 108% to 104%. Two other customers show similar risk patterns — combined exposure including Acme would be $9,800/month."

Cross-Cutting AI Capabilities

Some AI capabilities span multiple layers rather than belonging to one:

AI-powered onboarding. The entire onboarding flow — connect systems, review entity matches, customize metrics, configure alerts — can be orchestrated by a conversational AI rather than a static wizard. "I've connected your HubSpot and pulled 347 companies. I also found your Stripe account has 312 customers. I've matched 267 of them automatically — that's about 85% coverage. There are 23 that I'm not sure about. Want to review those now, or should I show you what metrics are already available with the matches I'm confident about?" This makes onboarding feel like working with a knowledgeable consultant rather than filling out a form.

Self-healing pipelines. When a data sync fails — API rate limit hit, authentication expired, schema changed — AI can diagnose the failure, attempt automated recovery (retry with backoff, refresh OAuth token, adapt to schema change), and if it can't resolve automatically, explain the issue to the customer in plain language: "Your HubSpot sync failed this morning because HubSpot changed how they structure deal properties. I've adapted to the new format, but I want to confirm: the field previously called 'amount' now appears as 'hs_deal_amount.' Should I treat these as the same field?" This eliminates a major category of support tickets that would otherwise require engineering intervention.

Natural language configuration. Instead of clicking through settings screens to adjust alert thresholds or metric definitions, the customer can simply say: "I want to be alerted when any customer's CSAT drops below 3.5, not 3.0" or "We don't count refunds as revenue — can you exclude those from the Monthly Revenue metric?" The AI translates these instructions into the appropriate configuration changes across the semantic layer, alert rules, and entity state definitions. This is particularly valuable because configuration is where most SMB analytics tools lose their customers — the tool is powerful but too complicated to customize without technical knowledge.

AI Capability Roadmap by Phase

Not all of these capabilities should be built at once. They map to the same incremental build plan as the core product:

Phase	Behind-the-Scenes AI	Customer-Facing AI
Phase 1 (Weeks 1-6)	Source data profiling and completeness scoring. Volume consistency checks. Data quality anomaly detection on initial sync. Basic LLM narration for weekly briefing.	Data health summary after connection. Pipeline health view (green/yellow/red). Confidence tags on briefing numbers. Natural language entity match review. Weekly briefing generation.
Phase 2 (Weeks 7-14)	Learning from human match confirmations. Alert feedback calibration. Metric stability monitoring. Cross-system reconciliation checks.	Match coverage reporting. Downstream impact narration for quality issues. On-demand Q&A. Conversational deep-dives.
Phase 3 (Weeks 15-26)	Smart sync scheduling. Metric anomaly pre-computation. Derived state rule suggestion. AI root cause analysis for metric changes.	Natural language entity summaries. "What changed?" entity diffs. Metric relationship discovery. Data quality improvement coaching. Natural language configuration.
Phase 4 (Months 7-12)	Intelligent schema mapping for new connectors. Multi-step agent reasoning. Adaptive briefing content. Self-healing pipelines. AI remediation suggestions.	Scenario simulation. Proactive insight generation. AI-powered onboarding. Metric definition suggestion.

The principle: Phase 1 AI capabilities should be the ones that deliver the most visible customer value with the least engineering complexity — briefing generation, match review narration, and data health summaries. Deeper AI capabilities that require accumulated data or complex orchestration (adaptive briefings, self-healing pipelines, scenario simulation) are deferred until the core product is proven and the data flywheel has begun spinning.

PART 4: THE ARCHITECTURE

4.1 Technology Stack

Component	Technology	License	Rationale
Managed warehouse	Postgres (Supabase / Neon / AWS RDS)	Open-source	Simple, reliable, Cube-compatible. SMB volumes don't need Snowflake.
Data ingestion	Airbyte (open-source) or custom REST extraction	Open-source	Pre-built connectors for HubSpot, Stripe, QuickBooks. Handles OAuth, pagination, rate limiting, incremental sync.
Entity matching (MVP)	Custom Python — deterministic rules	Proprietary	Simple enough to own. Normalize + exact match + confidence scoring.

Component	Technology	License	Rationale
Entity matching (v2)	Splink (Python, DuckDB backend)	MIT	Proven probabilistic matching. Unsupervised. Fast. Add when deterministic isn't enough.
Semantic layer	Cube Core	Apache 2.0	Best open-source semantic layer. SQL + REST + GraphQL APIs, caching, Postgres-native.
AI agent	Claude / GPT API + custom orchestration	API	LLM for narrative generation, intent extraction, Q&A summarization.
Application backend	Node.js or Python (FastAPI)	—	API server, job scheduling, customer management.
Application frontend	Next.js (React)	MIT	Customer portal, onboarding flows, entity match review UI.
Application database	Postgres (separate from warehouse)	Open-source	Customer accounts, configuration, match confirmations.
Job scheduling	Temporal or simple cron	Open-source	Orchestrate data sync, entity matching, metric computation, briefing generation.

4.2 Data Flow Architecture

```
┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
│   HubSpot   │  │   Stripe   │  │  QuickBooks  │
│   (CRM)     │  │  (Billing) │  │ (Accounting) │
└──────────────────┘  └──────────────────┘  └──────────────────┘
     │ OAuth/API        │ OAuth/API        │ OAuth/API
     ▼                  ▼                  ▼

┌──────────────────────────────────────────────────────────────┐
│                  Data Ingestion Layer                        │
│              (Airbyte or Custom REST Extraction)             │
│          Handles: Auth, Pagination, Rate Limiting,           │
│          Incremental Sync, Schema Detection                  │
└──────────────────────────────────────────────────────────────┘
                    │
                    ▼

┌──────────────────────────────────────────────────────────────┐
│                Managed Warehouse (Postgres)                  │
│                                                              │
│  raw.hubspot_companies    raw.stripe_customers               │
│  raw.hubspot_contacts     raw.stripe_subscriptions           │
│  raw.hubspot_deals        raw.quickbooks_customers           │
│                           raw.quickbooks_invoices            │
│                                                              │
```

```
│ Connector modules normalize into:                  │
│ canonical.companies  canonical.invoices            │
│ canonical.contacts   canonical.subscriptions       │
│ canonical.deals      (system-agnostic)             │
└──────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────┐
│         Data Quality Engine                        │
│                                                    │
│ Source profiling: completeness, freshness, volume  │
│ Anomaly detection: schema changes, duplicates      │
│ Value validation: range checks, type checks        │
│                                                    │
│ Output: quality_profiles, quality_alerts,          │
│         confidence_scores per field/table          │
└──────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────┐
│         Entity Matching Engine                     │
│                                                    │
│ Operates on canonical.* tables                     │
│ Tier 1: Deterministic (email, domain, ID)          │
│ Tier 2: Probabilistic (Splink — post-MVP)          │
│ Tier 3: Human confirmation (onboarding UI)         │
│                                                    │
│ Output: entities + entity_links tables             │
│ Quality monitor: match coverage, confidence dist.  │
└──────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────┐
│       Transformation Layer (dbt or SQL)            │
│                                                    │
│ Joins canonical tables via entity_links into:      │
│ analytics.companies (unified)                      │
│ analytics.contacts (unified)                       │
│ analytics.deals (enriched)                         │
│ analytics.invoices (unified)                       │
│ analytics.subscriptions                            │
│ analytics.entity_states (derived)                  │
└──────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────┐
│         Semantic Layer (Cube Core)                 │
```

```
│                                   │
│ YAML data models → SQL generation → Caching        │
│ APIs: SQL (Postgres-compat) + REST + GraphQL       │
│ Template: B2B SaaS metrics (30+ definitions)       │
│ Quality monitor: metric reconciliation, stability  │
└──────────────┬──────────────────────────────────────┘
               │                  │
               ▼                  ▼

┌──────────────────────────┐  ┌──────────────────────────────────┐
│ AI Agent (Apollo)        │  │ BI Tools (Power BI,              │
│                          │  │ Tableau, Metabase, etc.)         │
│ Queries Cube API         │  │                                  │
│ Generates:               │  │ Query via SQL API                │
│ • Briefings              │  │ Same certified metrics           │
│ • Alerts                 │  │ as AI agent uses                 │
│ • Q&A responses          │  │                                  │
│ Quality: output          │  │                                  │
│ confidence tags          │  │                                  │
└──────────────────────────┘  └──────────────────────────────────┘
```

4.3 Data Modeling Philosophy: Traditional Approaches vs. DataOrbit

DataOrbit's transformation layer and semantic layer use the same foundational concepts as traditional data warehouse architecture. This section documents the relationship honestly — what's the same, what's different, what's enhanced, and what has been transformed from a consulting engagement into a product.

This matters for three audiences: engineers making schema design decisions need to know they should use proven modeling patterns rather than inventing new ones; prospects with technical backgrounds need honest answers about how DataOrbit relates to approaches they already know; and investors need to understand where the genuine innovation lives versus where we're applying established techniques.

What Is the Same: Shared Foundations

DataOrbit's data model shares the same conceptual foundations as traditional dimensional modeling (Kimball methodology), which has been the industry standard for analytics data modeling for over 30 years. Being direct about this prevents overclaiming and builds credibility with technical evaluators.

Fact and dimension concepts are preserved. DataOrbit's `analytics.invoices` table is a fact table — it contains measures (amount, date, status) and foreign keys to entity dimensions. The `analytics.companies` table is a dimension table — it contains entity attributes (name, segment, account manager, state). Whether expressed as a physical star schema or as Cube YAML definitions, the logical structure is the same: measurements connected to the entities they describe through join relationships.

The entity resolution tables are a conformed dimension. In Kimball's methodology, a conformed dimension is a dimension table that is consistent across multiple fact tables from different source systems, enabling cross-source analysis. DataOrbit's `entities` and `entity_links` tables — which link the same customer across HubSpot, Stripe, and QuickBooks so that facts from all three systems can be joined — are exactly this concept. The

terminology is different ("entity graph" vs. "conformed dimension") but the purpose is identical: create a single, consistent view of an entity that bridges multiple data sources.

The same modeling decisions still apply. Fact table grain selection (per invoice or per line item?), refund handling (negative amounts or separate fact?), currency conversion, null handling, outer joins for partial matches — these are dimensional modeling decisions that DataOrbit must make. The methodology for making them well (Kimball's four-step design process) is established and should be followed.

Slowly changing dimensions are still relevant. When a customer changes their plan tier, when an employee moves to a different team, when a company rebrands — the entity graph needs to handle historical versus current attribute values. DataOrbit's `analytics.entity_states` with timestamps is functionally equivalent to Kimball's SCD Type 2 (maintain history with effective dates). This is a solved problem, and DataOrbit uses the established solution.

Data quality problems are not solved by the abstraction. If QuickBooks has duplicate invoices, the semantic layer will double-count revenue. If HubSpot has contacts assigned to the wrong company, the entity graph will produce incorrect cross-system profiles. The semantic layer provides metric consistency (the same definition everywhere), not data correctness (the underlying data being accurate). This is true of traditional star schemas too — the modeling layer organizes data; it doesn't clean it.

What Is Different: Genuine Architectural Differences

Despite sharing foundational concepts, DataOrbit's approach has genuine structural differences from a traditional warehouse + star schema + BI tool architecture. These differences are real, defensible, and important — they are not marketing reframings of the same thing.

Difference 1: The schema is auto-generated from templates, not hand-designed per customer.

In a traditional warehouse, a data engineer designs the star schema as a bespoke project. They interview business stakeholders to understand metric definitions, map source system fields to the dimensional model, handle edge cases specific to that company's data, and document the design. This takes weeks to months per customer.

DataOrbit auto-generates the schema from pre-built templates when the customer connects their systems. The template for HubSpot + Stripe + QuickBooks defines which raw tables to extract, how to normalize them, how to structure the analytics tables, and what grain to use — all decided once during template development and deployed automatically for every customer on that combination.

Aspect	Traditional	DataOrbit
Who designs the schema	Data engineer, per customer	Product team, per template (once)
Design time	Weeks to months	Zero (template applied automatically)
Customer involvement	Heavy (interviews, reviews, iteration)	Light (review and confirm defaults)
Marginal cost per customer	$50K-$200K (consulting)	$0 (template reuse)
Customization	Fully bespoke	80% template, 20% guided configuration

Difference 2: Entity resolution is dynamic with feedback loops, not static ETL logic.

In a traditional warehouse, conformed dimensions are built during the ETL process using static matching logic written by a data engineer: "When the customer name in SAP matches the customer name in Salesforce after normalization, merge them." If the logic is wrong, or if a new edge case appears, the data engineer manually updates the ETL code.

DataOrbit's entity matching engine is dynamic. Deterministic matching runs automatically. Probabilistic matching suggests candidates. Human confirmations feed back into the system. Confirmed matches improve future matching for similar entities. The entity resolution layer evolves over time — it learns from customer feedback rather than relying solely on rules written during initial development.

Aspect	Traditional	DataOrbit
Matching logic	Static rules in ETL code	Tiered: deterministic → probabilistic → human-in-the-loop
Confidence tracking	None (match or no match)	Per-match confidence scores (high/medium/low)
Human feedback loop	None (engineer fixes manually)	Built-in review UI; confirmations improve matching
Cross-customer learning	None (each customer is isolated)	Aggregate match patterns improve templates over time
Handling ambiguity	Fails silently or duplicates	Presents candidates for human review with explanation

Difference 3: Derived entity states are pre-computed and first-class.

In a traditional star schema, entity states are implicit — they're derived at query time by joining multiple fact tables: "Is there an active subscription? Are there overdue invoices? Are there open escalations?" Every dashboard, report, or AI query that needs customer state must re-derive it through complex multi-table joins.

DataOrbit pre-computes entity states as a materialized concept. The `analytics.entity_states` table holds the current state, the state history, and the cross-system signals that triggered each state. Downstream consumers (semantic layer, AI agent, dashboards) read a single field rather than re-deriving through joins.

Aspect	Traditional	DataOrbit
Entity state	Implicit (derived at query time via joins)	Explicit (pre-computed, materialized, with history)
State definition	Ad-hoc per query or dashboard	Governed rules per entity type, defined in templates
State history	Not tracked (unless custom SCD implementation)	First-class: state, timestamp, triggering signals
Cross-system derivation	Requires analyst to write correct multi-table joins	Automatic: rules evaluate across all connected systems
Downstream complexity	Every consumer re-derives state differently	Every consumer reads the same pre-computed state

Difference 4: The semantic layer abstracts away SQL entirely, enabling AI integration.

In a traditional star schema, consumers query the schema using SQL. BI tools generate SQL. Analysts write SQL. AI systems — if they exist — must either generate SQL (hallucination-prone) or have SQL pre-written for them.

DataOrbit's semantic layer (Cube Core) sits between the dimensional model and all consumers. Consumers request named business concepts ("MRR," "pipeline coverage," "revenue per employee") and Cube generates the SQL. No consumer — human, BI tool, or AI agent — ever needs to know the underlying schema.

This abstraction is what makes the natural language Query Pipeline possible. The AI agent doesn't generate SQL against a star schema. It maps the user's question to a named metric in the semantic layer, asks Cube for that metric, and narrates the result. The semantic layer constrains the AI's vocabulary to valid, pre-defined business concepts — eliminating the most common source of AI hallucination in analytics.

Aspect	Traditional	DataOrbit
Consumer interface	SQL against star schema	Named metrics via semantic layer API
SQL knowledge required	Yes (analyst, BI developer, or AI-generated)	No (Cube generates SQL from YAML definitions)
AI integration	AI must generate SQL (error-prone) or use pre-written queries	AI queries by metric name; Cube handles SQL generation
Metric consistency	Depends on discipline (different queries can define metrics differently)	Enforced (one YAML definition serves all consumers)
Caching	BI tool-specific or custom	Built-in (Cube Store) — sub-second for repeated queries

Difference 5: The entire stack is vertically integrated into one product.

In a traditional architecture, the customer assembles five or more products:

```
Traditional: Warehouse (Snowflake) + ETL (Fivetran) + Transformation (dbt)
             + Semantic Layer (maybe) + BI Tool (Tableau) + AI (maybe)
             = 5-6 products, 5-6 vendors, integrated by the customer's data team

DataOrbit:   One product. Warehouse + Connectors + Entity Matching
             + Semantic Layer + AI Agent = vertically integrated.
             No assembly required.
```

This isn't an architectural difference in the data modeling sense — it's a product architecture difference. But for the target customer (no data team, no warehouse, no BI tool), it's the most important difference. They don't need to evaluate, purchase, integrate, and maintain five separate products. They connect their tools and get intelligence.

What Is Enhanced: Improvements Over Traditional Approaches

Beyond the structural differences, several aspects of DataOrbit's approach are enhancements of traditional techniques — doing something established but doing it better for the specific use case.

Enhancement 1: Metric definitions are version-controlled and auditable.

In a traditional BI deployment, metric definitions often live in the BI tool's configuration (a Tableau calculated field, a Power BI DAX expression) or in analyst-written SQL queries saved in personal folders. They drift over time. Nobody remembers exactly how "Net Revenue Retention" was defined six months ago.

DataOrbit's metric definitions live in Cube YAML files that are version-controlled. Every change to a metric definition is tracked. A customer can see exactly how "MRR" was computed on any given date, what changed, and when. This auditability is important for board reporting, investor communication, and regulatory compliance — all of which are relevant for the ICP.

Enhancement 2: Cross-system metrics are pre-defined per system combination.

Traditional approaches require per-customer metric engineering. "Revenue Per Employee" requires joining ERP data with HRIS data, which means a data engineer has to understand both systems' schemas, handle the joins, and write the metric definition.

DataOrbit ships pre-built cross-system metric definitions per system combination. The B2B SaaS template (HubSpot + Stripe + QuickBooks + BambooHR) includes "Revenue Per Employee" already defined, already tested, and ready to compute on day one. The engineering work happened once during template development. Every customer on that combination gets the metric for free.

Enhancement 3: Confidence scores propagate from entity matching through to metric reporting.

Traditional conformed dimensions are binary — a record either matches or it doesn't. There's no concept of "we're 70% sure these are the same customer."

DataOrbit's entity graph carries confidence scores on every match, and these scores propagate into the semantic layer and AI agent. A cross-system metric computed from a medium-confidence entity match is flagged: "Customer health score: 65 (match confidence: medium — based on company name similarity, not confirmed by email or domain)." This transparency is something traditional architectures don't provide, and it directly addresses the trust problem that plagues AI-powered analytics.

Enhancement 4: The data model supports both current state and diagnostic reasoning.

A traditional star schema is optimized for answering "what happened?" — aggregating facts across dimensions and time periods. It answers questions like "what was revenue last quarter?" well. It answers "why did revenue drop?" poorly, because the diagnostic requires traversing relationships between metrics (revenue → pipeline → conversion rate → rep activity → capacity).

DataOrbit's influence maps (metric decomposition trees) are pre-built into the data model. When a metric moves, the system knows which related metrics to check and in what order. This structured diagnostic capability is layered on top of the dimensional model, not a replacement for it — but it's an enhancement that traditional star schemas don't include.

What Has Been Productized: Consulting vs. Product

This is the most important distinction in the entire architecture, and it is where DataOrbit's genuine innovation lives. The underlying data modeling concepts are not new. What is new is transforming them from consulting deliverables into product artifacts.

What a traditional data warehouse project looks like (consulting model):

Phase	Activities	Duration	Cost	Who Does It
Discovery	Interview stakeholders, document data sources, map business processes, define requirements	2-4 weeks	$15K-$40K	Data consultants
Schema design	Design dimensional model, define fact/dimension tables, handle edge cases, document grain decisions	3-6 weeks	$20K-$60K	Data engineer
ETL development	Build extraction pipelines for each source system, transformation logic, data quality checks	4-8 weeks	$30K-$80K	Data engineer
Entity resolution	Define matching rules for conformed dimensions, handle duplicates, build golden records	2-4 weeks	$15K-$40K	Data engineer + business analyst
Metric definitions	Define each metric with business stakeholders, implement in BI tool or semantic layer	2-4 weeks	$10K-$30K	BI developer + business analyst
BI development	Build dashboards, reports, configure alerts	3-6 weeks	$20K-$50K	BI developer
Testing and validation	Reconcile metrics against source systems, user acceptance testing	2-3 weeks	$10K-$20K	Data engineer + business users
Training and handoff	Train users, document everything, transition to maintenance	1-2 weeks	$5K-$15K	Consultants
TOTAL		**6-12 months**	**$125K-$335K**	**3-5 specialists**

After this project, the customer has a working analytics stack. But they also have an ongoing maintenance burden: pipeline monitoring, schema updates when source systems change, metric definition updates as the business evolves, dashboard maintenance, and infrastructure management. This typically requires a part-time or full-time data engineer ($120K-$180K/year) or a managed services contract ($3K-$10K/month).

What DataOrbit productizes from the above:

Consulting Phase	DataOrbit Productization	How It Works
Discovery	Eliminated. System combination detected at OAuth connection time.	Customer connects tools. DataOrbit identifies which systems are connected and selects the appropriate template.
Schema design	Pre-built templates per system combination.	The dimensional model for HubSpot + Stripe + QuickBooks is a product artifact — designed once, shipped to every customer on that combination.
ETL development	Airbyte connectors + custom extraction pipelines (productized).	Pre-built, pre-tested connectors handle extraction. Transformation logic is template-driven.
Entity resolution	Entity matching engine with deterministic rules, probabilistic matching, and human-in-the-loop review (productized).	Matching runs automatically on connection. Uncertain matches presented for human review through a guided UI.
Metric definitions	Template metric libraries per system combination (productized).	30+ certified metrics ship with each template. Customer reviews and confirms rather than defines from scratch.
BI development	AI-generated briefings + Cube SQL API for external BI tools (productized).	The Monday briefing replaces custom dashboards for most use cases. Cube's API serves BI tools that want custom views.
Testing and validation	Automated reconciliation + transparency features (productized).	Confidence scores, match coverage reporting, metric lineage — built into the product, not a consulting phase.
Training and handoff	Conversational onboarding + self-service (productized).	AI-guided onboarding replaces consultant-led training.

The cost comparison:

Dimension	Traditional (Consulting)	DataOrbit (Product)
Upfront cost	$125K-$335K	$0 (subscription model)
Time to first insight	6-12 months	Days
Ongoing maintenance	$36K-$120K/year (engineer or managed services)	Included in subscription ($6K-$42K/year at Starter)
Total Year 1 cost	$160K-$455K	$6K-$42K
Staff required	3-5 specialists during build; 1 ongoing	0 (CEO connects tools and receives intelligence)
Customization	100% bespoke	80% template + 20% guided configuration
Scales across customers	Each customer is a new project	Each customer on the same system combination is a template deployment

What CANNOT be productized (and shouldn't be claimed):

Not everything in the consulting model can be eliminated. DataOrbit should be honest about what still requires human judgment:

- **Business context that differs between companies.** Two companies running HubSpot + QuickBooks may define "revenue" differently (one includes professional services, the other doesn't). The template provides a sensible default; the customer must review and adjust. This is a 30-minute configuration step, not a 6-week project — but it's not zero.
- **Edge cases in entity matching.** The templates handle standard matching patterns. Companies with unusual naming conventions, multiple entities using the same email domain (holding companies), or merged/acquired entities will need human review. The product facilitates this; it doesn't eliminate it.
- **Custom metrics beyond the template.** The 80% that's template-covered is genuinely 80% for the target ICP. The remaining 20% requires guided configuration or, for complex cases, professional services support. DataOrbit should offer this as a paid add-on for Growth and Scale tier customers.
- **Data quality remediation.** If the customer's source data is messy (duplicates in HubSpot, miscategorized invoices in QuickBooks), DataOrbit can detect and surface the issues but cannot fix them — the fixes need to happen in the source systems. Transparency about data quality is a product feature; data cleanup is still the customer's responsibility.

How to Talk About This

To engineers: "Use Kimball's dimensional modeling methodology for the physical schema. Fact tables, dimension tables, conformed dimensions, SCD Type 2 for entity history. The templates are implementations of Kimball methodology for specific system combinations. Don't reinvent data modeling — productize it."

To technical prospects: "The underlying data model uses proven dimensional modeling principles — the same approach that powers analytics at companies 100× your size. The difference is that you don't design it, build it, or maintain it. It ships pre-configured for your system combination, with certified metrics and an AI analyst on top."

To non-technical prospects: "You connect your tools. DataOrbit automatically organizes all your data, links your customers and employees across every system, and gives you certified business metrics and an AI analyst — in days, not months. No data team required."

To investors: "The traditional analytics stack costs $125K-$335K to build and requires 6-12 months and a team of specialists. DataOrbit delivers the same outcome as a $500/month product that deploys in days. The unit economics work because each system combination template is built once and deployed to every customer on that combination at zero marginal cost. The template library is the compounding asset."

4.4 Template Architecture: How Pre-Built Intelligence Actually Works

The blueprint claims that DataOrbit ships "pre-built templates per system combination" as a core differentiator. This section pressure-tests that claim, identifies where it breaks down, introduces the composable architecture that makes it viable, and documents how AI augments every layer to handle the real-world variation that no static template can cover.

This section matters because if the template claim doesn't hold up to scrutiny, the entire cost advantage over traditional consulting collapses. The analysis below is honest about what works, what doesn't, and what requires AI to bridge the gap.

The Problem With Monolithic Templates

The original claim implied a monolithic template per system combination: "When a customer connects HubSpot + Stripe + QuickBooks, they get a pre-built template." The math on this approach doesn't work.

Consider the supported systems across five categories:

- CRMs: HubSpot, Salesforce, Zoho, Pipedrive, Close — 5 systems
- Accounting: QuickBooks, Xero, NetSuite, FreshBooks, Sage — 5 systems
- Billing: Stripe, Chargebee, Recurly, Paddle — 4 systems
- HRIS: BambooHR, Gusto, Deel, Rippling — 4 systems
- Support: Zendesk, Freshdesk, Intercom, Help Scout — 4 systems

Unique combinations for a five-system stack: 5 × 5 × 4 × 4 × 4 = 1,600. Even for the MVP three-system stack (CRM + Billing + Accounting): 5 × 4 × 5 = 100 combinations.

Building and maintaining 100 monolithic templates is not feasible for a startup. Each template would contain its own entity matching rules, its own metric definitions, its own derived state logic — all of which would need to be updated independently when a bug is found or a metric definition is improved.

The concept of pre-built intelligence is sound. The monolithic implementation is not. The answer is composable templates augmented by AI.

The Composable Template Architecture

Instead of monolithic templates per system combination, DataOrbit uses a layered composition of independent modules that snap together. Each layer has a specific scope, scales linearly (not combinatorially), and can be independently developed, tested, and improved.

Level 1: System Connector Modules (one per system)

Each supported SaaS system has a single connector module that defines: how to authenticate (OAuth flow), which API endpoints to call, how to map the system's fields to DataOrbit's canonical data model, and how to handle pagination, rate limits, and incremental sync.

The HubSpot connector knows that HubSpot calls them "Companies" with a field called "domain." The Salesforce connector knows that Salesforce calls them "Accounts" with a field called "Website." Both map to the same internal concept: `canonical.companies.domain`.

This is per-system, not per-combination. Supporting 22 systems requires 22 connector modules. Linear scaling.

Level 2: The Canonical Data Model (one, shared across all systems)

This is the critical architectural piece that makes composability possible. Instead of mapping each system directly to the analytics layer, DataOrbit defines a canonical data model — an abstract schema representing business concepts independent of any specific system:

```
canonical.companies      (any company/customer/account from any system)
canonical.contacts       (any person/contact from any system)
canonical.deals          (any opportunity/deal from any CRM)
canonical.invoices       (any invoice/charge from any accounting/billing system)
canonical.subscriptions  (any recurring relationship from any billing system)
canonical.tickets        (any support interaction from any support system)
canonical.employees      (any team member from any HRIS)
```

Each canonical table has a standardized set of fields representing the universal attributes of that entity type:

```
canonical.companies:
  - entity_id        (DataOrbit internal ID)
  - name             (normalized company name)
  - domain           (primary web domain)
  - industry         (standardized industry classification)
  - employee_count   (if available)
  - address_city
  - address_country
  - source_system    (which SaaS system this record came from)
  - source_system_id (the ID in the source system)
  - created_at
  - updated_at

canonical.invoices:
  - invoice_id        (DataOrbit internal ID)
  - company_source_id (source system's customer/company ID — linked to entity_id via entity_links after matching)
  - amount            (normalized to base currency)
  - status            (standardized: draft / sent / paid / overdue / void)
  - invoice_date
  - due_date
  - paid_date
  - source_system
  - source_system_id
```

Every connector module maps its system's fields to this canonical model. The downstream layers — entity matching, semantic layer, AI agent — only ever see the canonical model. They never see system-specific schemas.

Level 3: Entity Matching Rules (against canonical model, not per system pair)

Entity matching operates on canonical fields. The core matching rules become system-agnostic:

"Match `canonical.companies` records where `source_system = 'crm'` against `canonical.companies` records where `source_system = 'accounting'` using: exact email match, normalized domain match, and normalized company name match."

This single rule set works whether the CRM is HubSpot or Salesforce and whether the accounting system is QuickBooks or Xero — because the connector modules already mapped everything to canonical fields.

The variation between system pairs is handled through field availability metadata. Each connector module declares which canonical fields are typically populated for that system:

```python
```

```
HUBSPOT_FIELD_AVAILABILITY = {
    "companies.domain": 0.78,      # 78% of records have domain
    "companies.name": 0.99,        # Almost all have a name
    "contacts.email": 0.92,        # Most contacts have email
    "companies.address_city": 0.45  # Less than half have address
}

QUICKBOOKS_FIELD_AVAILABILITY = {
    "companies.domain": 0.05,      # QuickBooks rarely has domain
    "companies.name": 0.99,        # Always has display name
    "contacts.email": 0.60,        # Some customers have email
    "companies.address_city": 0.70  # Most have billing address
}
```

The entity matching engine reads these profiles and adjusts its strategy per customer: "For this customer on HubSpot + QuickBooks, domain matching will be effective for HubSpot records but nearly useless for QuickBooks. Weight email matching and name matching higher for this pair."

This is one set of matching rules with adaptive field weighting — not a separate rule set per system pair.

Level 4: Metric Definitions (against canonical model, system-agnostic)

Metric definitions reference canonical tables, not system-specific tables:

```yaml
# This metric works for ANY accounting system because it
# queries canonical.invoices, not quickbooks.invoices
cubes:
  - name: revenue
    sql_table: canonical.invoices
    measures:
      - name: total_revenue
        sql: "SUM(amount)"
        type: sum
        filters:
          - sql: "status = 'paid'"
      - name: mrr
        sql_table: canonical.subscriptions
        sql: "SUM(recurring_amount)"
        type: sum
        filters:
          - sql: "status = 'active'"
```

"Revenue Per Employee" is: `canonical.invoices (SUM amount WHERE paid) / canonical.employees (COUNT WHERE active)`. This definition works whether the invoices come from QuickBooks, Xero, or NetSuite, and whether the

employees come from BambooHR, Gusto, or Rippling. The connector modules handle the mapping; the metric definition is system-agnostic.

This means: DataOrbit doesn't need 100 metric libraries for 100 system combinations. It needs one metric library per vertical that operates on the canonical model.

Level 5: Vertical Templates (industry-specific metric bundles)

On top of the system-agnostic metric library, vertical templates bundle metrics, derived states, alert thresholds, and briefing formats for specific industries:

- **B2B SaaS template:** MRR, NRR, churn rate, pipeline velocity, customer health score, briefing emphasizing subscription metrics
- **Professional Services template:** utilization rate, project profitability, revenue per consultant, briefing emphasizing capacity and margin
- **E-commerce template:** CAC by channel, repeat purchase rate, inventory turnover, briefing emphasizing customer acquisition and retention

These templates select which metrics to activate and set sensible defaults — but they don't redefine the underlying computations, because those are already system-agnostic.

Concrete Example: Two Companies, Different Systems, Same Outcome

Company A: TechStartup Inc. CRM: HubSpot | Billing: Stripe | Accounting: QuickBooks Online 120 employees, B2B SaaS, $8M ARR

Company B: CloudServices Co. CRM: Salesforce | Billing: Chargebee | Accounting: Xero 200 employees, B2B SaaS, $15M ARR

Under a monolithic approach, these would need entirely separate templates. Under the composable architecture:

What's different (connector modules only):

Canonical Field	Company A (HubSpot)	Company B (Salesforce)
companies.name	hubspot.companies.name	salesforce.accounts.Name
companies.domain	hubspot.companies.domain	salesforce.accounts.Website
companies.industry	hubspot.companies.industry	salesforce.accounts.Industry
companies.owner	hubspot.companies.hubspot_owner_id	salesforce.accounts.OwnerId

Canonical Field	Company A (Stripe)	Company B (Chargebee)
`subscriptions.recurring_amount`	`stripe.subscriptions.plan.amount`	`chargebee.subscriptions.mrr`
`subscriptions.status`	`stripe.subscriptions.status`	`chargebee.subscriptions.status`
`subscriptions.cancel_date`	`stripe.subscriptions.canceled_at`	`chargebee.subscriptions.cancelled_at`

Canonical Field	Company A (QuickBooks)	Company B (Xero)
`invoices.amount`	`qbo.invoices.TotalAmt`	`xero.invoices.Total`
`invoices.status`	`qbo.invoices.Balance` (0 = paid)	`xero.invoices.Status`
`invoices.due_date`	`qbo.invoices.DueDate`	`xero.invoices.DueDateString`

What's identical (everything downstream):

Both companies get the exact same:

- **Entity matching rules:** Match `canonical.companies` across source systems using email, domain, and normalized name. Rules don't reference HubSpot or Salesforce — they reference canonical fields.
- **Metric definitions:** "MRR" = `SUM(canonical.subscriptions.recurring_amount) WHERE status = 'active'`. Works identically for Stripe and Chargebee data because both were mapped to `canonical.subscriptions.recurring_amount`.
- **Derived entity states:** "At Risk" = active subscription + declining engagement OR open escalation OR overdue invoice. References canonical fields only.
- **Weekly briefing:** Same structure, same sections, same AI prompt templates. Only the numbers and entity names differ.
- **Alert thresholds:** Same defaults. "Pipeline coverage below 2.5×" works the same regardless of CRM source.

The result: DataOrbit builds 6 connector modules (one per system) and 1 vertical template (B2B SaaS). Both companies get a fully functional intelligence layer. The marginal cost of supporting Company B after Company A is already live: three new connector modules. Zero changes to entity matching, metrics, states, briefings, or alerts.

What Breaks Down: Honest Assessment of the 20% Gap

The composable architecture handles approximately 80% of the work cleanly. The remaining 20% is where real-world variation creates friction that cannot be solved by static templates alone.

Gap 1: Field availability varies across systems in the same category.

HubSpot populates "Company Domain" for ~78% of records. QuickBooks has no equivalent field. Salesforce has "Website" but it's often empty. The canonical `companies.domain` field has different completeness depending on the CRM.

Impact: Entity matching rules that rely on domain matching work well for HubSpot customers (strong signal) and poorly for Pipedrive customers (weak signal). The rules are the same, but effectiveness varies by system.

Gap 2: Business semantics differ between systems in the same category.

QuickBooks has "Customers" that can be companies or individuals — there's no structural distinction. HubSpot has "Companies" and "Contacts" as separate objects. Salesforce has "Accounts" and "Contacts" with an explicit relationship. Pipedrive has "Organizations" and "People."

Impact: Mapping to `canonical.companies` is clean for HubSpot and Salesforce (explicit company object) but ambiguous for QuickBooks (a "Customer" named "John Smith" — person or sole proprietorship?).

Gap 3: Custom fields and non-standard workflows.

A company tracking "Product Usage Score" as a HubSpot custom property can't benefit from a pre-built metric referencing it, because DataOrbit doesn't know the custom field exists until the customer connects. Standard fields map cleanly. Custom fields require discovery.

Impact: The 80% of metrics using standard fields work out of the box. The 20% depending on customer-specific custom fields require configuration.

Gap 4: Metric definition nuances between companies.

"Revenue" seems universal, but one company includes professional services revenue and another doesn't. One counts refunds as negative revenue; another tracks them separately. One recognizes revenue at invoice creation; another at payment receipt.

Impact: The template's default "Total Revenue" may not match what the customer considers "revenue." If the customer accepts without reviewing, their metrics will be subtly wrong.

How AI Closes the 20% Gap

This is where DataOrbit genuinely differentiates: using AI not as a standalone feature but as the mechanism that makes templates adaptive, self-improving, and accurate despite real-world variation.

AI-Augmented Connector Development: Accelerating New System Support

When DataOrbit adds support for a new system (say, Pipedrive CRM), the traditional approach is: an engineer studies the API documentation, writes field mappings, handles edge cases, and tests. This takes 1-2 weeks per connector.

The AI-augmented approach: provide the AI with Pipedrive's API schema (field names, types, descriptions) and the canonical model definition. The AI generates a draft mapping: "Pipedrive `organizations.name` → `canonical.companies.name`, Pipedrive `deals.value` → `canonical.deals.amount`." The engineer reviews and corrects edge cases.

This works because schema mapping is a pattern-matching problem. After the AI has seen how HubSpot, Salesforce, and Zoho map to the canonical model, it recognizes the same concepts in Pipedrive's schema. Field

names like "name," "email," "amount," "status," "created_at" are near-universal across SaaS APIs.

Acceleration curve: the first connector (HubSpot) is mostly manual. The second (Salesforce) is ~50% AI-suggested. By the tenth connector, the AI generates 80-90% of the mapping correctly, and the engineer's job becomes review and correction.

AI-Augmented Entity Matching: Adaptive Field Weighting

Instead of hard-coding match weights per system pair, AI learns optimal weights from confirmed matches across the customer base.

After 50 customers on HubSpot + QuickBooks, the system has observed thousands of match confirmations and rejections. AI analyzes: "For HubSpot + QuickBooks, email match is the strongest signal (93% precision). Name match after normalization is moderate (78% precision). Address matching adds almost no signal (54% precision). Recommended weights: email 0.9, name 0.6, address 0.1."

These learned weights are stored per system pair and applied automatically for new customers. This directly addresses Gap 1 (field availability variation) without requiring per-system-pair rules — the weights adapt based on what actually works.

AI-Augmented Canonical Mapping: Custom Field Discovery

When a customer connects, AI scans for custom fields and infers their purpose — addressing Gap 3:

"We found 4 custom properties in your HubSpot that aren't part of the standard schema:

1. `nps_score` (number, 0-10) — Appears to be Net Promoter Score. Can include in Customer Health Score.
2. `contract_end_date` (date) — Appears to be a renewal date. Can trigger renewal risk alerts.
3. `implementation_stage` (dropdown: kickoff/in-progress/complete) — Appears to track onboarding. Can derive onboarding states.
4. `internal_notes` (text) — Free-text notes. Won't include in metrics but can surface in entity summaries.

Would you like to activate any of these?"

This bridges the 80% template coverage and the 20% customer-specific configuration. The AI infers the mapping; the customer confirms with a click — replacing what would otherwise be a consulting conversation.

AI-Augmented Semantic Mapping: Business Entity Classification

For Gap 2 (QuickBooks' ambiguous "Customer" entity), AI applies classification heuristics and improves them over time:

QuickBooks customer "John Smith" → AI checks: no business suffix, matches "FirstName LastName" pattern, no associated employees → classifies as `canonical.contacts` (person).

QuickBooks customer "Smith & Associates LLC" → AI checks: contains business suffix "LLC" → classifies as `canonical.companies`.

QuickBooks customer "The Johnson Group" → AI checks: no clear suffix, but "Group" is a business indicator → classifies as `canonical.companies` with medium confidence → flags for human review.

After 50 customers on QuickBooks, the AI has seen thousands of these classifications confirmed or rejected. Its accuracy improves: "On QuickBooks, 'The [Name] Group' pattern has been confirmed as a company 94% of the time. Auto-classify as company."

AI-Augmented Metric Reconciliation: Discrepancy Detection

For Gap 4 (metric definition nuances), AI detects discrepancies automatically after the initial data load — addressing the problem before the customer even notices:

"We computed your Monthly Revenue two ways:

- From QuickBooks paid invoices: $487,000
- From Stripe successful charges: $412,000
- Difference: $75,000

The difference appears to come from 12 QuickBooks invoices categorized as 'Professional Services' that don't have corresponding Stripe charges. This suggests revenue from both subscriptions (Stripe) and services (QuickBooks-only).

How would you like to define 'Monthly Revenue'?
→ All revenue (subscriptions + services): $487,000
→ Subscription revenue only: $412,000
→ Track both separately"

This replaces the traditional consulting conversation ("How do you define revenue?") with an AI-driven discovery that finds the actual answer in the customer's data. The customer doesn't need to know their data well enough to anticipate the issue — the AI finds it for them.

AI-Augmented Briefings: Adaptive Emphasis

The weekly briefing template defines which sections and metrics to include. AI makes the narrative adaptive based on what's actually happening in the data:

In a normal week: balanced coverage across Financial, Pipeline, Customers, Team sections.

In a week where two major customers entered At Risk: "Two of your top 20 customers entered At Risk this week. This section has been expanded with full context for both."

In a week where pipeline coverage dropped below 2×: "Pipeline needs immediate attention. Coverage dropped to 1.6× — the lowest in 6 months. The rest of the business is tracking normally."

The template defines what CAN be in the briefing. AI decides what SHOULD be emphasized based on the actual data. This is something a static template can never do — it requires understanding what's noteworthy this specific week for this specific customer.

The Revised Template Claim

Based on this analysis, the original claim ("pre-built template per system combination") is revised to:

DataOrbit uses a **composable template architecture** built on a **canonical data model**. Each SaaS system has a connector module that maps its data to the canonical model. Metric definitions, entity matching rules, and

derived states operate on the canonical model — making them system-agnostic. A customer connecting any combination of supported CRM + billing + accounting systems gets the same certified metrics, the same entity matching, and the same AI analyst.

AI augments every layer of the composition: accelerating new connector development through schema pattern recognition, optimizing entity matching weights from cross-customer observations, discovering and mapping custom fields automatically, detecting metric definition discrepancies in the customer's actual data, and adapting briefing emphasis based on what matters most each week.

What scales linearly: Connector modules (one per system). Supporting 22 systems requires 22 modules.

What scales once: The canonical data model, the metric library per vertical, the entity matching rules, the derived state logic, the briefing templates, and the alert thresholds. Built once, deployed to every customer.

What improves with scale: AI-learned field weights, custom field classification accuracy, metric reconciliation patterns, and entity matching precision — all compound across the customer base.

What requires per-customer work (the honest 20%): Custom field activation, metric definition review and adjustment, entity match confirmation for ambiguous cases, and edge cases in business entity classification. This is a 30-minute guided configuration step, not a 6-week consulting project — but it is not zero.

What This Means for Engineering Priorities

Priority	Action	Why
Build the canonical model first	Define the canonical schema for companies, contacts, deals, invoices, subscriptions, tickets, employees. This is the foundation everything else builds on.	If the canonical model is wrong, every connector module, every metric, and every matching rule is affected. Get this right.
Build connector modules for MVP systems	HubSpot, Stripe, QuickBooks connectors mapping to the canonical model.	These serve the first customers and validate the canonical model against real data.
Build metric definitions against canonical model	Write the B2B SaaS metric library referencing canonical tables only.	Validates that system-agnostic metrics produce correct results from system-specific data.
Defer connector proliferation	Don't build Salesforce, Xero, or Chargebee connectors until the canonical model is proven.	If the canonical model needs restructuring after real customer data, it's cheaper to change before 10 connectors depend on it.
Invest in AI augmentation early	Build custom field discovery and metric reconciliation from the start.	These directly address the 20% gap that determines whether customers trust the product. The AI doesn't need to be sophisticated — even basic pattern matching on field names adds significant value.

4.5 Key Technical Risks and Mitigations

Risk	Severity	Description	Mitigation
Entity matching accuracy	High	False positives contaminate all downstream metrics	Lead with deterministic matching (high precision). Human confirmation for uncertain matches. Confidence scores propagate to metrics.
API rate limits and reliability	Medium	SaaS APIs have rate limits, downtime, and schema changes	Use Airbyte's battle-tested connectors. Implement retry logic. Daily sync reduces API pressure. Layer 2 schema stability monitoring detects changes automatically.
Semantic layer complexity	Low	Cube Core is well-documented and production-proven	Embed Cube Core; don't build from scratch. Focus engineering on templates, not engine.
AI hallucination	Medium	LLM may generate incorrect summaries of metric data	Ground all AI output in Cube's certified metrics. Every number comes from a governed SQL query, not from LLM generation. LLM only narrates — it doesn't compute. Layer 2 confidence tags on every output make reliability transparent.
Data quality in SMB systems	High	SMB SaaS data is messy — duplicates, missing fields, inconsistent entry	Layer 2 (Data Quality Engine) profiles sources, detects anomalies, flags duplicates, and traces quality issues to downstream business impact. Confidence scores propagate to every metric and AI output.
Multi-tenant isolation	Medium	Customer data must be strictly isolated	Separate Postgres schemas or databases per customer. Row-level security. Cube's multi-tenancy support.
Data quality alert fatigue	Medium	Too many quality alerts desensitize customers, undermining trust	Rank quality issues by downstream business impact, not technical severity. Suppress low-impact alerts. Learn from customer feedback which alerts are actionable.
Confidence score miscalibration	Medium	If confidence tags are inaccurate (marking unreliable data as high confidence), trust is destroyed faster than having no confidence tags at all	Validate confidence scoring against manual reconciliation for early customers. Conservative defaults: when uncertain, mark as medium rather than high confidence.

4.6 AI Implementation Architecture

Section 3.5 describes what AI capabilities exist at each layer. This section describes how they are built — the

engineering patterns and architecture that make AI features reliable, affordable, and hallucination-resistant.

The fundamental principle: **the LLM is the voice, not the brain.** DataOrbit's database, metric definitions, entity graph, and business rules are the brain. The LLM's only job is to translate between human language and structured systems — in both directions. Human words come in, structured queries go to DataOrbit's systems, structured results come back, and the LLM turns those results into human words.

This principle eliminates the three most common failures of AI product development: excessive cost (from sending too much data to the LLM), hallucination (from asking the LLM to generate facts), and context management complexity (from trying to make the LLM remember everything).

The Four-Component Pattern

Every AI-powered interaction in DataOrbit — briefings, Q&A, entity summaries, match review narration, conversational configuration — follows the same four-component pattern:

```
User input (natural language)
        ↓
┌──────────────────────────────┐
│ 1. INTENT ROUTER             │
│                              │
│ Determines WHAT the          │
│ user wants.                  │
│ Cheap: small model,          │
│ classifier, or rules.        │
└──────────────────────────────┘
        ↓
┌──────────────────────────────┐
│ 2. CONTEXT FETCHER           │
│                              │
│ Queries DataOrbit's          │
│ own database to get          │
│ the specific data            │
│ relevant to the intent.      │
│ No LLM. SQL queries.         │
└──────────────────────────────┘
        ↓
┌──────────────────────────────┐
│ 3. ACTION EXECUTOR           │
│                              │
│ If the user wants to         │
│ change something, code       │
│ executes the change.         │
│ Deterministic. No LLM.       │
│ Skipped for questions.       │
└──────────────────────────────┘
        ↓
┌──────────────────────────────┐
│ 4. RESPONSE WRITER           │
│                              │
│ LLM takes RESULTS from       │
│ steps 2-3 and writes a       │
│ human-readable response.     │
│ Narrates. Never invents.     │
└──────────────────────────────┘
```

The critical insight: only component 4 requires an LLM, and it receives only the specific, pre-fetched data it needs to compose a response. It never accesses the database directly, never invents facts, and never decides what to do. The other three components are conventional code — classifiers, SQL queries, and business logic.

How Each Component Works

Component 1: Intent Router

The Intent Router classifies the user's message into a predefined intent category and extracts structured parameters. It answers two questions: "What type of thing does the user want?" and "What specific entities or topics are they asking about?"

DataOrbit's intent categories are finite and well-defined:

Intent	Example Input	Extracted Parameters
`entity_inquiry`	"What's happening with Acme Corp?"	entity_name: "Acme Corp"
`metric_question`	"What's our current MRR?"	metric: "mrr", period: "current"
`diagnostic_question`	"Why is Sarah's team behind?"	subject: "Sarah's team", topic: "pipeline"
`comparison`	"How does this month compare to last?"	metric: context-dependent, periods: "this_month", "last_month"
`config_change`	"Alert me when CSAT drops below 3.5"	alert_type: "csat_threshold", value: 3.5
`match_confirmation`	"Yes, those are the same company"	action: "confirm_match"
`general_followup`	"Tell me more about that"	reference: previous topic

Implementation options, from simplest to most capable:

Option A — Keyword rules (simplest, good enough for MVP): If the message contains a company name from the entity graph, classify as `entity_inquiry`. If it contains a metric name ("MRR," "pipeline," "revenue"), classify as `metric_question`. If it starts with "why" or "how come," classify as `diagnostic_question`. If it contains "alert," "notify," or "tell me when," classify as `config_change`. This is a function with if/else branches. No ML required. Covers 70-80% of real user messages for a product where the domain vocabulary is constrained.

Option B — Small classifier model (better, moderate effort): Train a lightweight text classifier (a few hundred labeled examples) using a service like OpenAI's fine-tuning or a local model like a sentence transformer. Input: user message. Output: intent category + confidence score. This handles natural phrasing variation better than keyword rules ("How's Acme doing?" vs. "Give me the status on Acme Corp" vs. "Anything I should know about Acme?"). Training data comes from the onboarding team's conversations with early customers.

Option C — Small LLM call with structured output (most flexible): Use a cheap, fast model (Claude Haiku or equivalent) with a system prompt that forces structured JSON output:

```
System: Classify the user's message into one of these intents:
entity_inquiry, metric_question, diagnostic_question,
comparison, config_change, match_confirmation, general_followup.

Extract parameters: entity_name, metric, period, subject, topic.

Respond ONLY with JSON. No other text.

User: "Why is Sarah's team behind on pipeline?"
```

Response:

```
json

{
  "intent": "diagnostic_question",
  "subject": "Sarah's team",
  "topic": "pipeline"
}
```

Cost: ~$0.0001 per classification using Haiku. This is the recommended approach for DataOrbit because the domain is narrow enough that Haiku handles it well, and it's far more flexible than keyword rules as the product evolves.

Component 2: Context Fetcher

The Context Fetcher is a set of code functions — one per intent type — that query DataOrbit's own systems to assemble exactly the data needed for the response. There is no LLM in this component. It is conventional backend code.

The key design principle: **each intent maps to a predefined set of queries.** The Intent Router's output determines which queries run. The queries are written in advance and parameterized by the extracted entities and topics.

Intent	Queries Executed
`entity_inquiry`	1. Look up entity in `entities` table by name. 2. Fetch entity profile (all cross-system attributes via `entity_links`). 3. Fetch current entity state and state history. 4. Fetch recent activity (last 30 days).
`metric_question`	1. Query Cube SQL API for the specified metric, current value. 2. Query historical values for trend context. 3. Fetch metric definition (plain-language description).
`diagnostic_question`	1. Identify the relevant top-level metric. 2. Query each driver in the influence map decomposition. 3. Fetch entity/team data for the subject. 4. Fetch HRIS data if people-related.
`comparison`	1. Query Cube for the metric across both specified periods. 2. Compute delta and percentage change. 3. Fetch notable events in the comparison window.
`config_change`	1. Look up current configuration for the setting being changed. 2. Validate the proposed new value against business rules.

Each of these is a function in the codebase that takes the Intent Router's extracted parameters, runs SQL queries (against Postgres directly or through Cube's API), and returns a structured data object. For example, the `entity_inquiry` context fetcher:

```python
```

```python
def fetch_entity_context(entity_name: str) -> dict:
    # Step 1: Find the entity
    entity = db.query(
        "SELECT * FROM entities WHERE display_name ILIKE %s",
        [f"%{entity_name}%"]
    )

    # Step 2: Fetch cross-system profile
    profile = db.query(
        "SELECT * FROM analytics.companies WHERE entity_id = %s",
        [entity.entity_id]
    )

    # Step 3: Fetch current state
    state = db.query(
        "SELECT * FROM analytics.entity_states WHERE entity_id = %s",
        [entity.entity_id]
    )

    # Step 4: Fetch recent activity
    activity = db.query("""
        SELECT source_system, event_type, event_date, description
        FROM analytics.entity_activity
        WHERE entity_id = %s AND event_date > NOW() - INTERVAL '30 days'
        ORDER BY event_date DESC LIMIT 10
    """, [entity.entity_id])

    return {
        "entity": entity,
        "profile": profile,
        "state": state,
        "recent_activity": activity
    }
```

This is the most important component in the architecture. It is what prevents hallucination — the LLM can only narrate what these queries return. If the data isn't in the database, it doesn't reach the LLM, and the LLM can't invent it.

Component 3: Action Executor

For intents that change something (config_change, match_confirmation), the Action Executor is conventional code that performs the change — updating a database row, modifying a Cube configuration, toggling an alert threshold. It is deterministic: the same input always produces the same change. No LLM is involved.

```python
```

```python
def execute_config_change(change_type: str, params: dict) -> dict:
    if change_type == "alert_threshold":
        old_value = db.query(
            "SELECT threshold FROM alert_rules WHERE metric = %s",
            [params["metric"]]
        )
        db.execute(
            "UPDATE alert_rules SET threshold = %s WHERE metric = %s",
            [params["new_value"], params["metric"]]
        )
        return {
            "action": "threshold_updated",
            "metric": params["metric"],
            "old_value": old_value,
            "new_value": params["new_value"]
        }
```

The Action Executor returns a structured result describing what was done, which the Response Writer then narrates.

Component 4: Response Writer

The Response Writer is the only component that uses an LLM. It receives the structured output from the Context Fetcher and/or Action Executor, along with a tightly constrained prompt template, and generates a natural language response.

The prompt template is specific to each intent type:

```
# Entity Inquiry prompt template:

System: You are a business analyst for DataOrbit. Summarize the
provided entity data in clear, concise business language.
Rules:
- Use ONLY the data provided below. Do not add any information.
- If a field is missing or null, say that data is not available.
- Lead with the most important finding.
- Cite which source system each data point comes from.
- Keep the response to 3-5 sentences unless the data warrants more.

Entity data:
{context_fetcher_output}

User's question: {original_user_message}
```

The constraint phrases — "Use ONLY the data provided," "Do not add any information," "If a field is missing, say that data is not available" — are what eliminate hallucination. The LLM is a formatting engine operating on

pre-fetched, pre-validated data.

Conversation State Management

Conversational continuity (users saying "Tell me more about that" or "What about their support tickets?") is handled by the application, not by the LLM.

Conversation state is a database record:

```sql
CREATE TABLE conversation_state (
    conversation_id UUID PRIMARY KEY,
    customer_id UUID,
    current_entity_id UUID,
    current_topic VARCHAR(100),
    current_metric VARCHAR(100),
    last_intent VARCHAR(50),
    pending_confirmation JSONB,
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);
```

When the user says "What about their support tickets?", the application code resolves the pronoun "their" by reading `current_entity_id` from the conversation state — not by asking the LLM to remember. The Intent Router classifies the intent, the application code substitutes the resolved entity, and the Context Fetcher queries for that entity's support tickets. The LLM never needs to "remember" anything.

Sliding context window for conversational tone:

To maintain natural conversational flow, the Response Writer receives the last 3-5 turns of conversation history alongside the fresh context data. This is enough for the LLM to write responses that feel connected ("As I mentioned, Acme's tickets are...") without sending the entire history. The conversation history is stored in a simple table:

```sql
CREATE TABLE conversation_turns (
    turn_id UUID PRIMARY KEY,
    conversation_id UUID,
    role VARCHAR(20),     -- 'user' or 'assistant'
    content TEXT,
    intent VARCHAR(50),
    turn_number INTEGER,
    created_at TIMESTAMP
);
```

The application fetches the last N turns and includes them in the LLM prompt. For conversations longer than 5-7 turns, the application code generates a brief summary of the conversation so far (this summary can itself be

generated by a cheap LLM call or by concatenating the intent + entity from each turn) and substitutes it for the full history.

Cost Model

Understanding the cost structure is essential for product pricing and architecture decisions:

Component	Technology	Cost per Interaction
Intent Router (Haiku)	Claude Haiku API	~$0.0001
Context Fetcher	SQL queries to Postgres / Cube	~$0.0000 (infrastructure cost only)
Action Executor	Application code	~$0.0000 (infrastructure cost only)
Response Writer (Haiku)	Claude Haiku API	~$0.0002 - $0.001
Response Writer (Sonnet, complex)	Claude Sonnet API	~$0.003 - $0.01
Total per Q&A interaction		**~$0.001 - $0.01**
Weekly briefing generation	Sonnet (longer output)	**~$0.01 - $0.03**

At these costs, a customer generating 100 Q&A interactions per month and 4 weekly briefings costs DataOrbit approximately $1-$2/month in LLM API fees. Even at 10× this estimate for safety margin, the AI cost per customer is negligible relative to the $500-$12,000/month subscription price.

Model selection rule: Use the cheapest model that produces acceptable output quality for each component. Intent routing and simple entity summaries: Haiku. Weekly briefings and diagnostic explanations: Sonnet. Only use the most capable model for complex multi-step reasoning if and when that feature is built.

Concrete Example: End-to-End Q&A Flow

To illustrate the complete pattern, here is the full flow for a real user interaction:

User says: "Why is Sarah's team behind on pipeline?"

Step 1 — Intent Router (Haiku, ~$0.0001):

```json
{
  "intent": "diagnostic_question",
  "subject": "Sarah's team",
  "topic": "pipeline"
}
```

Step 2 — Context Fetcher (SQL queries, ~$0.00):

```
Query 1: SELECT * FROM analytics.teams WHERE manager_name ILIKE '%sarah%'
→ Team: Sarah Martinez, 3 direct reports (Mike, Alex, Jordan)

Query 2: SELECT rep_name, pipeline_value, deals_count
        FROM cube.pipeline_by_rep WHERE team_id = 'team_sarah'
→ Mike: $320K (12 deals), Alex: $85K (4 deals), Jordan: $210K (9 deals)
  Team total: $615K vs $820K target

Query 3: SELECT rep_name, pipeline_value FROM cube.pipeline_by_rep
        WHERE period = 'last_month' AND team_id = 'team_sarah'
→ Last month: Mike $290K, Alex $205K, Jordan $195K. Total: $690K

Query 4: SELECT employee_name, pto_days_this_month
        FROM analytics.employees WHERE team_id = 'team_sarah'
→ Alex: 8 PTO days this month. Mike: 0. Jordan: 1.
```

Step 3 — Action Executor: Skipped (this is a question, not a change).

Step 4 — Response Writer (Haiku or Sonnet, ~$0.001-$0.003):

Prompt:

```
System: You are a business analyst. Explain the pipeline
situation based ONLY on the data below. Do not speculate
beyond the data.

Data:
- Team: Sarah Martinez, 3 reps
- Current pipeline: Mike $320K (above avg), Alex $85K
  (well below avg), Jordan $210K (normal)
- Team total: $615K vs $820K target (75% of target)
- Last month: Mike $290K, Alex $205K, Jordan $195K
- PTO this month: Alex took 8 days off
- The $205K gap is almost entirely Alex's decline
  ($205K → $85K, a $120K drop)

User question: "Why is Sarah's team behind on pipeline?"
```

Response: "Sarah's team is at $615K against an $820K target — about 75% of plan. The gap is almost entirely driven by Alex, whose pipeline dropped from $205K last month to $85K this month. Alex was out for 8 of the last 20 business days, which explains the drop — fewer active days means fewer deals generated. Mike is actually performing above his usual level ($320K vs $290K last month), and Jordan is tracking normally. The issue appears to be a temporary capacity gap from Alex's time off, not a performance problem."

Every fact came from the database. The LLM identified the pattern in the pre-fetched data and explained it clearly.

The Query Pipeline: How Natural Language Becomes a Database Answer

The entire AI interaction model in DataOrbit follows a single, repeatable pipeline. Understanding how each transition in this pipeline works — how the output of one stage becomes the input to the next — is the most important engineering concept in the product.

The pipeline:

```
Natural Language In → Known Vocabulary → SQL Queries → Structured Results → Natural Language Out
```

There are five stages and four transitions between them. Each transition is a different type of operation, uses different technology, and has different failure modes. This section documents each transition in detail.

Stage 1: Natural Language In

This is the raw user input. It can be anything the user types:

- "How is Acme Corp doing?"
- "What's our MRR?"
- "Why is Sarah's team behind on pipeline?"
- "How much money are we making this quarter?"
- "Alert me when any customer's CSAT drops below 3.5"
- "Tell me more about that"

The user's phrasing is unpredictable. They use pronouns ("their," "that"), synonyms ("money" instead of "revenue"), incomplete sentences, follow-up references, typos, and ambiguous language. This stage is pure unstructured text.

Transition 1: Natural Language → Known Vocabulary

What happens: The system translates the user's unconstrained natural language into the constrained vocabulary of DataOrbit's data model. This is the most critical transition in the pipeline because it is where ambiguity is resolved, and everything downstream depends on getting it right.

What "known vocabulary" means: Before any user types anything, DataOrbit already knows every word that matters. The known vocabulary is not the English language — it is the finite set of objects in the customer's data model:

Vocabulary Type	Source	Example Terms	Approximate Count
Entity names	`entities` table	"Acme Corp," "TechFlow Inc," "Sarah Martinez"	Hundreds (varies per customer)
Metric names	Cube YAML definitions	"MRR," "pipeline coverage," "churn rate"	30-50
Metric aliases	Maintained alias dictionary	"monthly recurring revenue" = MRR, "pipe" = pipeline	60-150
Entity states	Entity state definitions	"healthy," "at risk," "churning," "overloaded"	8-12
Entity types	Data model	"customer," "company," "employee," "deal"	4-6
Dimensions	Cube dimension definitions	"by team," "by rep," "by quarter," "by segment"	15-25
Time periods	Time parsing rules	"this month," "last quarter," "YTD," "last 90 days"	10-15
Intent patterns	Intent classification	"why," "how much," "alert me," "compare"	10-15
Configuration targets	Settings schema	"alert threshold," "sync frequency," "briefing schedule"	5-10

Total known vocabulary: roughly 300-500 meaningful terms per customer, every one of which maps to a specific object in the data model.

How the translation works — three mechanisms operating together:

Mechanism A: Direct vocabulary lookup. The system scans the user's message for exact or near-exact matches against the known vocabulary. This is string matching against a dictionary, not AI.

"What's our **MRR**?" → "MRR" matches metric alias dictionary → resolves to metric_id: `monthly_recurring_revenue`.

"How is **Acme Corp** doing?" → "Acme Corp" matches `entities` display_name column → resolves to entity_id: `acme_corp_uuid`.

"**Alert me** when CSAT drops below 3.5" → "Alert me" matches intent pattern → resolves to intent: `config_change`.

Implementation: dictionary lookup + substring matching. Cost: zero. Speed: microseconds. Reliability: 100% for exact matches.

Mechanism B: Fuzzy matching. When the user's phrasing doesn't exactly match the vocabulary — typos, abbreviations, partial names — a fuzzy string matching library finds the closest match above a confidence

threshold.

"What's happening with **Techflow**?" → fuzzy match against entity names → "TechFlow Inc" at 91% similarity → resolves to entity_id: `techflow_uuid`.

"How's our **revnue per head**?" → fuzzy match against metric aliases → "revenue per employee" at 87% similarity → resolves to metric_id: `revenue_per_employee`.

Implementation: rapidfuzz or similar library. Cost: zero. Speed: microseconds. Reliability: high for known vocabulary with minor variations. Fails gracefully — if no match exceeds the confidence threshold (e.g., 80%), the system knows it didn't understand and can ask for clarification.

Mechanism C: LLM-assisted classification. When the user's phrasing doesn't contain recognizable vocabulary at all — "How much money are we making?" contains neither "revenue" nor "MRR" — a small, cheap LLM call maps the natural language to the known vocabulary.

The critical design: the LLM is given the complete list of valid vocabulary terms and can only select from that list. It is not generating new terms. It is classifying.

```
System: Map the user's question to DataOrbit's known vocabulary.

Available metrics (pick from this list ONLY):
- monthly_recurring_revenue
- annual_recurring_revenue
- total_revenue
- weighted_pipeline
- churn_rate
- customer_satisfaction_score
- revenue_per_employee
- pipeline_coverage
[... complete list ...]

Available entity types: company, employee, deal, invoice
Available periods: current, last_month, last_quarter, ytd, custom

User message: "How much money are we making this quarter?"

Respond with JSON only:
```

LLM response:

```
json
```

```
{
  "intent": "metric_question",
  "metric": "total_revenue",
  "entity_filter": null,
  "period": "current_quarter"
}
```

The LLM cannot hallucinate a metric that doesn't exist because the output is constrained to the provided list. If the user asks about something DataOrbit doesn't track ("What's our brand sentiment?"), the LLM either maps it to the closest valid metric or returns "no matching metric" — it cannot invent a metric called `brand_sentiment` because that term isn't in the list.

Implementation: Claude Haiku with structured JSON output. Cost: ~$0.0001 per call. Speed: 200-500ms. Reliability: very high for domain-specific classification when the vocabulary list is provided.

How the three mechanisms work together:

```
User message arrives
      ↓
[Mechanism A: Direct lookup]
  Found exact entity name? → resolved
  Found exact metric alias? → resolved
  Found exact intent pattern? → resolved
      ↓ (for anything not resolved by A)
[Mechanism B: Fuzzy matching]
  Found entity name within 80% similarity? → resolved
  Found metric alias within 80% similarity? → resolved
      ↓ (for anything not resolved by A or B)
[Mechanism C: LLM classification]
  Send unresolved terms to Haiku with vocabulary list
  LLM maps to closest valid terms → resolved
      ↓
All terms resolved → proceed to Stage 2
Any terms unresolvable → ask user for clarification
```

Most messages (70-80%) are fully resolved by Mechanisms A and B alone — no LLM call needed. The LLM is the fallback for natural phrasing that doesn't contain recognizable vocabulary. This keeps average cost per interaction extremely low.

The output of Transition 1 is a structured object containing only known vocabulary terms:

```
json
```

```
{
  "intent": "diagnostic_question",
  "entities": [{"id": "sarah_martinez_uuid", "type": "employee", "name": "Sarah Martinez"}],
  "metrics": ["weighted_pipeline"],
  "period": "current_month",
  "comparison_period": null,
  "topic": "pipeline"
}
```

Every field references a concrete object in the data model. Nothing is ambiguous. This object is what the rest of the pipeline operates on.

Stage 2: Known Vocabulary

This is the structured representation of the user's intent, expressed entirely in data model terms. It is the pivot point of the pipeline — everything before it is about understanding the human; everything after it is about querying the database.

The known vocabulary object is what makes the system deterministic from this point forward. Given the same structured object, the system will always execute the same queries and return the same results (assuming the underlying data hasn't changed). The non-determinism of natural language is fully absorbed in Transition 1 and does not leak into the rest of the pipeline.

Transition 2: Known Vocabulary → SQL Queries

What happens: The system selects and parameterizes pre-written SQL query templates based on the intent and extracted parameters. This is conventional code — a function that reads the structured object from Transition 1 and constructs the appropriate database queries.

The key insight: the queries are pre-written. Every intent type has a corresponding set of SQL queries (or Cube API calls) that were written by engineers during product development. The system does not generate SQL at runtime. It selects from existing queries and fills in parameters.

How intent maps to queries — the query registry:

The query registry is a mapping from (intent, topic, entity_type) tuples to ordered lists of query templates:

```python
```

```
QUERY_REGISTRY = {

    ("entity_inquiry", None, "company"): [
        # Query 1: Fetch entity profile
        """SELECT e.*, es.state, es.state_since, es.state_reason
          FROM analytics.companies e
          JOIN analytics.entity_states es ON e.entity_id = es.entity_id
          WHERE e.entity_id = :entity_id""",

        # Query 2: Fetch financial summary
        """SELECT mrr, lifetime_revenue, outstanding_balance,
                days_since_last_payment
          FROM cube_api.customer_financials
          WHERE entity_id = :entity_id""",

        # Query 3: Fetch support summary
        """SELECT open_tickets, avg_csat, last_interaction_date
          FROM cube_api.customer_support
          WHERE entity_id = :entity_id""",

        # Query 4: Fetch recent activity
        """SELECT source_system, event_type, event_date, description
          FROM analytics.entity_activity
          WHERE entity_id = :entity_id
            AND event_date > NOW() - INTERVAL '30 days'
          ORDER BY event_date DESC LIMIT 10""",
    ],

    ("metric_question", None, None): [
        # Query 1: Current metric value
        """SELECT metric_value, metric_date, metric_target
          FROM cube_api.metrics
          WHERE metric_id = :metric_id
            AND period = :period""",

        # Query 2: Historical trend (last 6 periods)
        """SELECT period_label, metric_value
          FROM cube_api.metric_history
          WHERE metric_id = :metric_id
          ORDER BY period_start DESC LIMIT 6""",

        # Query 3: Metric definition for citation
        """SELECT display_name, description, formula_description,
                source_systems
          FROM metric_definitions
          WHERE metric_id = :metric_id""",
```

```
    ],

    ("diagnostic_question", "pipeline", "employee"): [
        # Query 1: Team identification
        """SELECT team_id, team_members
          FROM analytics.teams
          WHERE manager_entity_id = :entity_id""",

        # Query 2: Per-rep pipeline current period
        """SELECT rep_name, pipeline_value, deals_count, quota
          FROM cube_api.pipeline_by_rep
          WHERE team_id = :team_id AND period = :period""",

        # Query 3: Per-rep pipeline prior period (for comparison)
        """SELECT rep_name, pipeline_value
          FROM cube_api.pipeline_by_rep
          WHERE team_id = :team_id AND period = :comparison_period""",

        # Query 4: HRIS context for the team
        """SELECT employee_name, pto_days_this_period,
                start_date, status
          FROM analytics.employees
          WHERE team_id = :team_id""",

        # Query 5: Team targets
        """SELECT target_value, attainment_pct
          FROM cube_api.team_targets
          WHERE team_id = :team_id AND period = :period""",
    ],

    ("config_change", "alert_threshold", None): [
        # Query 1: Current configuration
        """SELECT metric, condition, threshold_value, enabled
          FROM alert_rules
          WHERE customer_id = :customer_id
            AND metric = :metric_id""",
    ],
}
```

How the selection works:

```python
```

```python
def select_queries(intent_object: dict) -> list:
    key = (
        intent_object["intent"],
        intent_object.get("topic"),
        intent_object["entities"][0]["type"] if intent_object.get("entities") else None
    )

    # Try exact match first
    if key in QUERY_REGISTRY:
        return QUERY_REGISTRY[key]

    # Fall back to less specific keys
    fallback_key = (intent_object["intent"], None, None)
    if fallback_key in QUERY_REGISTRY:
        return QUERY_REGISTRY[fallback_key]

    return []  # No matching queries — system can't answer this
```

Parameter substitution: The `:entity_id`, `:metric_id`, `:period`, and `:team_id` placeholders in the query templates are filled from the intent object's extracted parameters. This is standard parameterized SQL — safe against injection, deterministic, and fast.

Where Cube's API fits: Many of the queries above go through Cube's SQL API rather than directly to Postgres. This means the queries benefit from Cube's caching layer (Cube Store), its semantic validation (if a metric doesn't exist, the query fails cleanly rather than returning garbage), and its computed metric logic (Cube handles the joins and aggregations defined in the YAML model). The Context Fetcher doesn't need to know how "Revenue Per Employee" is computed — it just asks Cube for it, and Cube executes the correct SQL against Postgres.

The query registry is the product's analytical brain. When DataOrbit adds a new intent type, a new metric, or a new diagnostic pathway, the engineering work is: write the SQL queries, add them to the registry, and map the intent tuple. This is conventional backend development, not AI development.

Stage 3: Structured Results

This is the raw data returned from the database — rows, numbers, dates, strings. It is structured, precise, and factual. There is no interpretation yet. Example:

```json
```

```
{
  "team": {
    "manager": "Sarah Martinez",
    "members": ["Mike Chen", "Alex Rivera", "Jordan Park"]
  },
  "pipeline_current": {
    "Mike Chen": {"value": 320000, "deals": 12},
    "Alex Rivera": {"value": 85000, "deals": 4},
    "Jordan Park": {"value": 210000, "deals": 9},
    "team_total": 615000,
    "team_target": 820000
  },
  "pipeline_prior": {
    "Mike Chen": 290000,
    "Alex Rivera": 205000,
    "Jordan Park": 195000,
    "team_total": 690000
  },
  "hris_context": {
    "Alex Rivera": {"pto_days_this_month": 8},
    "Mike Chen": {"pto_days_this_month": 0},
    "Jordan Park": {"pto_days_this_month": 1}
  }
}
```

This data is ground truth. It came from the governed semantic layer and the entity graph. Every number is traceable to a source system. No AI has been involved since Transition 1.

Transition 3: Structured Results → Pre-Composed Context

What happens: Before the results reach the LLM, application code transforms the raw structured data into a human-readable context block — a text summary of the data that the LLM will narrate. This is an often-overlooked step but it is critical for controlling the LLM's output quality.

Why not send the raw JSON to the LLM? Three reasons:

First, raw JSON wastes tokens. The JSON structure above is verbose — field names, nesting, data types. Converting it to a concise text summary reduces token count (and therefore cost and latency) significantly.

Second, raw JSON introduces interpretation ambiguity. If the LLM sees `"value": 320000`, it might write "$320,000" or "320K" or "three hundred and twenty thousand." If it sees `"pto_days_this_month": 8`, it doesn't know whether 8 is a lot or a little without context. Pre-composing the context lets your code add the interpretation: "Alex: 8 PTO days (40% of working days this month)."

Third, pre-composition lets your code do the analytical heavy lifting. The LLM should not be doing arithmetic or comparisons. Your code computes: "Team total is 75% of target. The $205K gap is primarily driven by Alex, whose pipeline dropped $120K from last month." The LLM then narrates these pre-computed findings rather than trying to do the math itself (which LLMs sometimes get wrong).

The pre-composition function:

```python
```

```python
def compose_context(intent: str, results: dict) -> str:
    if intent == "diagnostic_question":
        lines = []
        current = results["pipeline_current"]
        prior = results["pipeline_prior"]
        hris = results["hris_context"]

        # Compute the analysis — this is YOUR code, not the LLM
        total = current["team_total"]
        target = current["team_target"]
        attainment = round(total / target * 100)
        gap = target - total

        lines.append(f"Team: {results['team']['manager']}, "
                     f"{len(results['team']['members'])} reps")
        lines.append(f"Team pipeline: ${total:,} vs ${target:,} "
                     f"target ({attainment}% of plan)")
        lines.append(f"Gap to target: ${gap:,}")
        lines.append("")

        # Per-rep breakdown with comparison
        for member in results["team"]["members"]:
            curr_val = current[member]["value"]
            prior_val = prior.get(member, 0)
            delta = curr_val - prior_val
            pto = hris.get(member, {}).get("pto_days_this_month", 0)

            direction = "up" if delta > 0 else "down"
            line = f"- {member}: ${curr_val:,} ({direction} " \
                   f"${abs(delta):,} from last month)"
            if pto > 3:
                line += f" — {pto} PTO days this month"
            lines.append(line)

        # Identify the primary driver
        deltas = {m: current[m]["value"] - prior.get(m, 0)
                  for m in results["team"]["members"]}
        biggest_decline = min(deltas, key=deltas.get)
        lines.append("")
        lines.append(f"Primary driver of gap: {biggest_decline} "
                     f"(${abs(deltas[biggest_decline]):,} decline)")

        return "\n".join(lines)
```

Output:

```
Team: Sarah Martinez, 3 reps
Team pipeline: $615,000 vs $820,000 target (75% of plan)
Gap to target: $205,000

- Mike Chen: $320,000 (up $30,000 from last month)
- Alex Rivera: $85,000 (down $120,000 from last month) — 8 PTO days this month
- Jordan Park: $210,000 (up $15,000 from last month)

Primary driver of gap: Alex Rivera ($120,000 decline)
```

This is what the LLM receives. It is concise, pre-analyzed, and unambiguous. The LLM's job is now trivial: turn this into a conversational paragraph. It cannot get the numbers wrong because the numbers are pre-computed. It cannot miss the key finding because the key finding is pre-identified.

Transition 4: Pre-Composed Context → Natural Language Out

What happens: The LLM receives the pre-composed context block, a prompt template specific to the intent type, and optionally the last few turns of conversation history. It generates a natural language response.

The prompt template:

```
System: You are a business analyst for DataOrbit. Your job is to
explain business data clearly and concisely.

Rules:
1. Use ONLY the data provided in the Context block below.
2. Do not add any information not present in the Context.
3. If something is unclear or data is missing, say so.
4. Cite source systems where relevant.
5. Lead with the most important finding.
6. Keep the response to 3-5 sentences for simple questions,
   or a structured summary for complex diagnostics.
7. Use specific numbers — do not round unless the user asks
   for a summary.

Context:
{pre_composed_context}

Recent conversation (last 3 turns):
{conversation_history}

User's question: {original_message}

Respond to the user's question based on the Context above.
```

Why this transition is reliable:

The LLM is operating under five constraints simultaneously:

1. **Input constraint** — it only sees the pre-composed context, not the full database
2. **Instruction constraint** — "Use ONLY the data provided" and "Do not add any information"
3. **Format constraint** — "3-5 sentences" or "structured summary"
4. **Vocabulary constraint** — the context uses DataOrbit's metric names and entity names, so the LLM naturally uses them in its response
5. **Verification constraint** — every number in the response can be traced back to the context block, which can be traced back to the SQL query, which can be traced back to the Cube metric definition, which can be traced back to the source system

If the LLM does hallucinate — which is rare under these constraints but not impossible — the hallucination is detectable. Every claim in the response should have a corresponding data point in the context block. A post-processing step can verify this (though for MVP, manual spot-checking is sufficient).

Stage 5: Natural Language Out

The final response delivered to the user. It is conversational, specific, and grounded:

"Sarah's team is at $615K against an $820K target — about 75% of plan. The gap is almost entirely driven by Alex Rivera, whose pipeline dropped from $205K last month to $85K this month. Alex was out for 8 days this month, which explains the decline — fewer working days means fewer deals generated. Mike is actually above his usual level ($320K, up $30K from last month), and Jordan is tracking normally at $210K. The issue appears to be a temporary capacity gap from Alex's time off, not a performance problem across the team."

Every number traces back through the pipeline:

- "$615K" → pre-composed context → SQL query against Cube → Cube metric `weighted_pipeline` → HubSpot deal data
- "8 days" → pre-composed context → SQL query against analytics.employees → BambooHR PTO data
- "$205K last month" → pre-composed context → SQL query against Cube (prior period) → HubSpot historical deal data

This traceability is what makes the system trustworthy. The user can ask "Where does that $615K number come from?" and the system can answer: "It's the sum of weighted pipeline values for the three reps on Sarah's team, pulled from HubSpot deal data. The weighting uses your configured stage probabilities: Proposal at 40%, Negotiation at 70%, Verbal Commit at 90%."

Complete Pipeline Summary

```
Stage 1: Natural Language In
  "Why is Sarah's team behind on pipeline?"
        │
        ▼
Transition 1: NL → Known Vocabulary
  [Dictionary lookup + fuzzy matching + LLM classification]
  Resolves: intent, entities, metrics, periods, topics
  Cost: $0.00 (lookup) to $0.0001 (LLM fallback)
        │
        ▼
Stage 2: Known Vocabulary
  { intent: "diagnostic_question", entity: "sarah_uuid",
    topic: "pipeline", period: "current_month" }
        │
        ▼
Transition 2: Known Vocabulary → SQL Queries
  [Query registry lookup + parameter substitution]
  Selects pre-written queries based on intent tuple
  Cost: $0.00 (code execution)
        │
        ▼
Stage 3: Structured Results
  { team_total: 615000, target: 820000,
    per_rep: [...], hris: [...] }
        │
        ▼
Transition 3: Results → Pre-Composed Context
  [Application code: arithmetic, comparison, analysis]
  Computes deltas, identifies drivers, formats for LLM
  Cost: $0.00 (code execution)
        │
        ▼
Stage 4: Pre-Composed Context
  "Team pipeline: $615,000 vs $820,000 (75%)
   Primary driver: Alex Rivera ($120K decline, 8 PTO days)"
        │
        ▼
Transition 4: Context → Natural Language Out
  [LLM with constrained prompt + pre-composed context]
  Narrates the pre-analyzed findings in conversational prose
  Cost: $0.001-$0.003 (LLM call)
        │
        ▼
Stage 5: Natural Language Out
  "Sarah's team is at $615K against an $820K target..."
```

The total cost per interaction: $0.001 to $0.01. The LLM is involved in at most two steps (Transition 1 fallback and Transition 4 narration), and in many interactions only one (Transition 4). Everything else is your code, your database, your business logic — deterministic, fast, and free.

What Does NOT Need an LLM

Many capabilities described in Section 3.5 require no LLM at all:

Capability	Implementation	Why No LLM Needed
Alert threshold monitoring	SQL query + comparison against stored threshold	Pure arithmetic
Entity state derivation	Business rules evaluated against entity graph	Deterministic rule evaluation
Data quality anomaly detection	Statistical checks on sync results (record counts, null rates)	Statistical computation
Smart sync scheduling	Frequency analysis of change rates per data type	Time-series analysis
Metric computation	Cube Core SQL generation from YAML definitions	Existing open-source tool
Match staleness detection	Compare current entity attributes against stored match snapshot	Field-level comparison

Reserve the LLM for the two things it uniquely enables: understanding natural language input and generating natural language output. Everything else is conventional software engineering.

PART 5: THE BUILD PLAN

5.1 Philosophy: Incremental, Customer-Driven

We do not build the full six-layer platform before getting customers. We build the minimum stack that delivers the core value proposition — a Monday morning briefing that synthesizes cross-system intelligence — and iterate based on real customer data and feedback.

The principle: **each phase should produce a product that is useful on its own**, not just a stepping stone to the next phase.

5.2 Phase 1: The Week One Briefing (Weeks 1-6)

Goal: Get the first briefing in a customer's hands from HubSpot + Stripe + QuickBooks data.

Team assumption: 2-3 engineers (full-stack + data engineering). A solo founder should plan 10-14 weeks for equivalent scope. The timelines below assume a small, focused team shipping daily.

What we build:

- **Layer 1 (Connectors + Warehouse):** OAuth connectors for HubSpot, Stripe, QuickBooks Online. Postgres warehouse with raw schemas. Daily batch sync.
- **Layer 2 (Data Quality — foundational checks):** Source data profiling after each sync: completeness rates, record counts, freshness timestamps. Duplicate detection. Volume consistency checks. Quality alerts for anomalies (record count drops, schema changes). Confidence scores per source table.
- **Layer 3 (Entity Matching — deterministic only):** Normalize email, domain, company name. Match across the three systems. Produce `entities` and `entity_links` tables with confidence scores. Simple web UI for human confirmation of uncertain matches.
- **Layer 4 (Entity Graph — basic):** Unified company/customer view joining data via `entity_links`. Compute basic derived states (Healthy, At Risk, Churning) from available data.
- **Layer 5 (Semantic Layer — starter metrics):** Deploy Cube Core with ~15 core metrics for the B2B SaaS template. Split into single-system (always available) and cross-system (available when matched).
- **Layer 6 (AI Agent — briefing only):** Generate a weekly briefing by querying Cube's SQL API, then LLM-summarizing into narrative format. Email delivery. No alerts or Q&A yet. Include confidence tags on all reported numbers based on Layer 2 quality scores.

What we explicitly defer:

- Probabilistic (fuzzy) entity matching
- More than 3 system connectors
- Exception alerts
- On-demand Q&A
- BI tool integration
- Influence maps and pattern detection
- Custom metric configuration UI
- Slack integration

Definition of success: 3 paying customers receive a weekly briefing they find useful and are willing to pay $500/month for.

Target customers: B2B SaaS companies, 50-200 employees, using HubSpot + Stripe + QuickBooks. Find through founder networks, Product Hunt launch, LinkedIn outreach to CEOs.

5.3 Phase 2: Intelligence That Comes to You (Weeks 7-14)

Goal: Add proactive alerts and on-demand Q&A. Expand metric coverage.

What we build:

- **Exception alerts:** Rule-based alerts for pre-defined thresholds (customer enters At-Risk, revenue below pace, payment failure, escalation unresolved). Email + Slack delivery.

- **On-demand Q&A:** Chat interface for natural language questions. Agent maps question to Cube metrics, queries, and summarizes.
- **Expanded metrics:** Grow template from 15 to 30+ metrics. Add people metrics (requires BambooHR connector).
- **Fourth connector:** BambooHR (HRIS) for employee data.
- **Entity matching improvements:** Add probabilistic matching (Splink) for companies that didn't match deterministically. Improve human review UI.
- **Data quality — downstream impact narration:** AI-generated explanations tracing quality issues through the pipeline to quantified business impact ("83 companies missing domain → $47K revenue unmonitored"). Cross-system metric reconciliation checks (detecting discrepancies between accounting and billing data).

Definition of success: Customers actively use alerts and Q&A in addition to weekly briefings. Net Promoter Score > 50.

5.4 Phase 3: The Full Platform (Weeks 15-26)

Goal: Complete the product for Growth tier customers.

What we build:

- **Fifth connector:** Zendesk (support) for customer satisfaction and ticket data.
- **BI tool integration:** Expose Cube's SQL API for external BI tool connections.
- **Influence maps:** Metric decomposition trees for structured diagnostic reasoning.
- **Statistical alerts:** Anomaly detection using historical baselines (requires 4+ weeks of data).
- **Custom metric configuration:** Guided UI for customers to adjust metric definitions and thresholds.
- **Daily briefings:** Option for daily briefings in addition to weekly.
- **Multi-team briefings:** Separate briefings for Sales, CS, Finance with role-appropriate focus.
- **Data quality — AI root cause analysis:** When metrics change unexpectedly, AI traces backwards through the pipeline to identify whether the cause is a data quality issue, a business event, or a pipeline failure. Data quality improvement coaching: AI identifies highest-impact quality improvements ranked by business impact.

Definition of success: 20+ paying customers across Starter and Growth tiers. Monthly recurring revenue of $15K+.

5.5 Phase 4: Scale and Expand (Months 7-12)

Goal: Vertical expansion and enterprise readiness.

What we build:

- **Second vertical template:** Professional Services (HubSpot + Harvest + QuickBooks + BambooHR) or E-commerce (Shopify + Klaviyo + Gorgias + QuickBooks).

- **Connector marketplace:** 10+ connectors covering major systems per vertical.
- **Advanced entity matching:** LLM-assisted matching for edge cases. Cross-customer matching quality improvements from aggregate learnings.
- **Embedded analytics API:** For Scale tier customers who want DataOrbit intelligence in their own products.
- **SOC 2 compliance:** Required for Growth and Scale tier customers.
- **Data quality — self-healing and remediation:** AI-powered remediation suggestions ("18 missing domains can be inferred from contact emails — apply?"). Self-healing pipelines that auto-adapt to schema changes detected by Layer 2. Conversational data quality remediation flows.

PART 6: THE BUSINESS

6.1 Revenue Model

Tier	Target	What's Included	Price
Starter	50-100 employees, 2-3 systems	Up to 3 connectors, weekly briefing, basic alerts, up to 20 certified metrics, email delivery	$500-$1,000/mo
Growth	100-300 employees, 4-6 systems	Up to 6 connectors, daily + weekly briefings, full alert suite, 50+ metrics, Slack + Q&A, BI tool connector	$1,500-$3,500/mo
Scale	300-1,000 employees, 6+ systems	Unlimited connectors, custom metrics, influence maps, multi-team briefings, embedded analytics API, dedicated onboarding	$5,000-$12,000/mo

Pricing philosophy: Value-based, not seat-based. The value scales with company complexity (more systems, more entities, more cross-system questions), not with the number of people reading the briefing.

Founding customer pricing: $500/month flat for the first 10 customers, regardless of tier. In exchange: weekly feedback calls, tolerance for rough edges, testimonial commitment.

6.2 Go-to-Market Strategy

Phase 1: Founder-Led Sales (Months 1-6)

- Target 10 B2B SaaS CEOs through personal network and warm introductions
- Validate: Do they have the Monday spreadsheet ritual? Do they use HubSpot + Stripe + QuickBooks?
- Offer the Week One Briefing: "Connect your tools. Get your first business briefing in 48 hours."
- Price: $500/month founding customer pricing
- Success metric: 3 paying customers, at least 1 willing to do a case study

Phase 2: Content-Led Growth (Months 4-9)

- Publish the "Cross-System Blindness" framework as thought leadership
- Create comparison content: "Old Way vs. DataOrbit Way"
- Target keywords: "cross-system analytics SMB," "business intelligence without data team," "agentic data warehouse"
- Product Hunt launch when Phase 2 product is ready

Phase 3: Product-Led Growth (Months 7-12)

- Free trial: Connect 2 systems, see a sample briefing
- Self-serve onboarding for Starter tier
- Sales-assisted for Growth and Scale tiers

6.3 Key Metrics We Track

Metric	Target (Month 6)	Target (Month 12)
Paying customers	10	50
Monthly recurring revenue	$5,000	$40,000
Customer retention (monthly)	>90%	>95%
Time to first briefing	<48 hours from OAuth connection (for customers with <500 entities and clean source data; larger or messier datasets may take up to 1 week including entity match review)	<24 hours
Entity match coverage	>70% (deterministic)	>85% (deterministic + probabilistic)
Briefing engagement rate	>60% open rate	>70% open rate

6.4 The Moat

The moat is not any single layer. It is the vertical integration of all layers into one product and the template libraries that compound over time:

1. **Managed warehouse** — auto-provisioned, invisible to customer

2. **Data quality engine** — full-pipeline quality monitoring with downstream impact analysis and confidence propagation
3. **Entity matching engine** — cross-system identity resolution with confidence scoring
4. **Certified semantic layer** — governed metric definitions per system combination
5. **Derived state engine** — cross-system business rules
6. **Proactive AI analyst** — briefings, alerts, Q&A grounded in certified metrics
7. **Template libraries** — pre-built for industry × system combination

Each layer alone is buildable. None of these concepts — managed warehouses, entity resolution, semantic layers, derived states, AI summarization — are individually novel. Enterprise data teams have assembled versions of this stack for years at significant cost. The moat is not conceptual novelty. The moat is in assembling all six layers, making them work together seamlessly, pre-configuring them per system combination, and delivering the complete experience to companies that have never had a data team — in days instead of months. The innovation is in the productization and accessibility, not in the underlying data architecture theory.

The templates compound. Every customer on HubSpot + QuickBooks + Stripe validates and refines the templates for that combination. After 100 customers on that stack, the metric definitions, entity matching rules, and alert thresholds become battle-tested — creating a data network effect that new entrants can't easily replicate.

Appendix A: Honest Limitations

Limitation	Reality	Mitigation
Data freshness	Default daily sync = up to 24-hour lag	Hourly sync for critical events. Timestamp all data in briefings.
Entity matching accuracy	Deterministic matching covers ~60-70%. Fuzzy matching adds risk of false positives.	Lead with high-precision deterministic matching. Human confirmation for uncertain matches. Confidence propagation to all downstream metrics.
Cross-system metrics require entity matching	Metrics like "customer health" don't work until entities are linked	Product works on day one with single-system metrics. Cross-system intelligence lights up progressively.
AI can misinterpret data	LLM may generate misleading narrative from correct numbers	Every AI response cites source systems and metric definitions. Human always makes final decision. LLM narrates; it doesn't compute.
SMB data quality is variable	Missing fields, duplicates, inconsistent entry are common	Layer 2 (Data Quality Engine) profiles every source after each sync, detects anomalies, flags duplicates, and traces quality issues to downstream business impact. Confidence scores propagate to every metric and AI output. Customers see a Pipeline Health view showing

Limitation	Reality	Mitigation
		data reliability at a glance. Quality issues are ranked by business impact, not technical severity.
System coverage	Only 3-5 connectors at launch	Prioritize by ICP frequency. Add connectors based on demand.
Influence map patterns are correlational	Empirical patterns are NOT causal claims	Transparent confidence labels. Never presented as proven causation.
Alert noise in early weeks	Statistical baselines need 2-4 weeks to calibrate	Suppress statistical alerts until baseline established. Quick feedback loop for relevance.

Appendix B: Competitive Quick-Reference

Competitor	Includes Warehouse	Entity Matching	Semantic Layer	Pre-Built Templates	Proactive AI	Target Buyer	Price
DataOrbit AI	Yes (managed, invisible)	Yes (cross-system)	Yes (Cube Core)	Yes (industry + system)	Yes (briefings + alerts + Q&A)	CEO/COO at 50-500 emp	$500-$12K/mo
Snowflake	Yes (BYO, self-managed)	No	Partial	No	No	Data engineering teams	$$$$
Panoply	Yes (managed)	No	No	No	No	SMB data teams	~$400-$2K/mo
ThoughtSpot	No (BYO required)	No	Yes	No	Partial (reactive)	Enterprise analytics	~$2,750+/mo
Narrative BI	No (API-only)	No	No	No	Yes (proactive)	SMB marketing/ops	~$100-$500/mo
Databox	No (API cache)	No	No	Partial (single-system)	Partial (alerts)	SMB marketing	$72-$500+/mo

Appendix C: Documents Referenced

This product blueprint synthesizes and supersedes the following working documents produced during the strategic planning process:

1. **DataOrbit_AI_Platform_Feature_Analysis.md** — Original product vision and architecture (input document)
2. **DataOrbit_AI_Competitive_Analysis_v2.md** — Comprehensive competitive landscape with corrected warehouse positioning
3. **DataOrbit_AI_Technical_Critique.md** — Critical technical analysis of entity matching and semantic layer feasibility
4. **DataOrbit_AI_Marketing_Page.md** — Marketing website content blueprint
5. **DataOrbit_Competitive_Map.jsx** — Interactive competitive positioning visualization

Appendix D: Future Work — Sections to Be Documented

The following topics have been identified as important but are not yet documented in this blueprint. They should be addressed in future revisions before external use of this document with investors or technical partners.

Security, Compliance, and Data Governance (HIGH PRIORITY)

DataOrbit handles financial data (QuickBooks), HR data including compensation (BambooHR), customer data (HubSpot), and payment data (Stripe). A dedicated section should cover: encryption at rest and in transit, data residency options for multi-region customers, access controls and audit logging, GDPR/CCPA data subject rights (specifically: how does "right to deletion" work when data is copied from source systems into the warehouse?), data retention policies, and how the pattern intelligence flywheel's cross-customer aggregate learning works without exposing individual customer data. Many ICP companies are subject to SOC 2 requirements from their own customers. Security posture is a sales gate, not a nice-to-have.

Multi-Tenant Architecture Patterns (HIGH PRIORITY)

The blueprint lists multi-tenant isolation as a risk but doesn't document the architecture decision. Database-per-customer is safest but most expensive. Schema-per-customer is a reasonable middle ground. Row-level security on shared tables is cheapest but riskiest. This choice directly impacts the pattern intelligence flywheel: cross-customer aggregate learning requires querying across customer boundaries, which is easy with shared tables and complex with database-per-customer. The tradeoffs should be documented with a clear recommendation.

Onboarding Experience Design (MEDIUM PRIORITY)

Onboarding is the product for the first hour — it's where trust is built or lost. The document describes steps (connect, confirm, customize, receive) but not the UX. How does the entity match review UI present ambiguous matches? How are data quality issues surfaced during onboarding vs. ongoing monitoring? Is the metric confirmation step a wizard, a checklist, or a conversational AI flow? The AI-first approach should extend to onboarding design, and the onboarding flow should be documented as a designed product experience, not just a list of steps.

Webhook and Real-Time Event Support (MEDIUM PRIORITY)

The architecture currently describes daily/hourly batch sync. Many SaaS systems support webhooks — Stripe fires webhooks on payment failures, HubSpot on deal stage changes, Zendesk on ticket escalations. Real-time event processing would make exception alerts significantly more timely (a customer whose Stripe payment fails

at 2 PM shouldn't wait until the overnight sync for DataOrbit to notice). This should be documented as a Phase 2 or 3 capability, with Phase 1 batch-only acknowledged as a limitation.

Data Retention and Storage Cost Scaling (MEDIUM PRIORITY)

DataOrbit stores all historical data in Postgres. For a customer with 5 connected systems syncing daily for 2 years, data volume grows continuously. The blueprint should address: storage cost per customer over time, data archival or tiering strategies (hot/warm/cold), whether historical data older than a certain age should be aggregated rather than stored at full granularity, and how this affects the pricing model. At $500/month for Starter tier, Postgres storage cost must remain a small fraction of revenue per customer. This needs validation with projected data volumes.

Pricing Validation Against Buyer Psychology (LOW PRIORITY)

Narrative BI is $100-$500/month. Databox is $72-$500+. DataOrbit's Starter tier begins at $500 — at the top of competitor ranges. The ICP currently spends $0-$500/month on analytics. The pricing section should explicitly connect to the ROI calculation: DataOrbit replaces 3-8 hours of CEO time per week (valued at $100-$300/hour for the ICP), making $500/month a clear return even at the low end. This framing should be documented in the pricing section.

Conversational Data Quality Remediation (LOW PRIORITY)

The Data Quality Engine detects issues and narrates impact, but there's no interactive remediation flow documented. A natural AI extension: "I found 4 potential duplicates in QuickBooks. Want to review them?" → AI shows candidates → customer confirms → DataOrbit records the decision and updates quality profiles. This would make the data quality layer feel like an AI-powered product feature, not just a monitoring dashboard. Should be documented as a Phase 3 or 4 capability.

"First Agentic Data Warehouse" Claim Validation (LOW PRIORITY)

The positioning statement claims DataOrbit is "the first agentic data warehouse built for companies that don't have a data team." This is defensible when specifically scoped to "first product that bundles warehouse + entity matching + semantic layer + AI agent for SMBs." However, competitors can approximate this by combining products (e.g., Mozart Data + Narrative BI). Before external use in marketing or investor materials, the "first" claim should be validated against the current competitive landscape at that time. The claim is strong but inherently fragile — it's true until someone else ships something similar.

This document is a living reference. It should be updated as product decisions are made, customer feedback is received, and market conditions change. Version history should be maintained.

Version 1.1 — February 18, 2026